An Experimental Investigation into Transformer Models Learning Integer Arithmetic

Master's Thesis

Submitted in partial fulfillment of
the requirements for the degree of
Diplom-Ingenieur

at the Faculty of Digital and Analytical Sciences
of the Paris-Lodron-University of Salzburg

by

Hugo Platzer

Supervisor: Ao.Univ.-Prof. Dipl.-Ing. Dr. Helmut Mayer
Department: Department of Artificial Intelligence
and Human Interfaces

Salzburg, December 2025

Abstract

Transformer networks have proven to be successful in a number of domains such as natural language translation, coding, and general question answering. Arithmetic tasks, however, can be challenging for them. We study the performance of small (around 10^7 parameters) transformer networks with a similar architecture to OpenAI GPT-2 on addition (3 to 10 decimal digits), multiplication (3 to 10 decimal digits), and integer square root (6 to 20 decimal digits). Training dataset sizes range from 10^3 to 10^5 samples, using character-level tokenization based on ASCII values. We observe good performance on addition but poor performance on multiplication, with the latter not being learnable at all with five or more operand digits, with integer square root being somewhere in the middle. Adjusting model dimensions, changing operand sampling, reversing the result and including a scratchpad are attempted to address the issues with multiplication and integer square root performance. While most strategies brought no dramatic improvement, scratchpads containing detailed intermediate computation steps showed promise in making unlearnable tasks like large multiplication learnable.

Contents

Abstract ii

1 Introduction 1
- 1.1 Natural language versus arithmetic . . . 1
- 1.2 Related work . . . 2

2 Transformer models 4
- 2.1 Types of transformer models . . . 4
- 2.2 Transformer decoder models . . . 4
- 2.3 Beyond text . . . 5
- 2.4 Tokenization . . . 5
- 2.5 Input and output . . . 6
- 2.6 Text generation . . . 6
- 2.7 User interaction . . . 7
- 2.8 Generative pre-training . . . 7
- 2.9 Attention mechanism . . . 7

3 OpenAI GPT-2 9
- 3.1 Achitecture . . . 9
- 3.2 Model size variants . . . 10
- 3.3 Tokenization . . . 10
- 3.4 Preprocessing . . . 11
- 3.5 Postprocessing . . . 12
- 3.6 Text generation . . . 13
- 3.7 Training . . . 14
- 3.8 GPT-2 block structure . . . 15
  - 3.8.1 Notation used . . . 16
  - 3.8.2 Layer normalization . . . 16
  - 3.8.3 Residual connection . . . 17
  - 3.8.4 Multi-head attention . . . 18
  - 3.8.5 Scaled dot-product attention . . . 20
  - 3.8.6 Multi-layer perceptron . . . 22

4 Arithmetic using GPT-2 23
- 4.1 Tokenization . . . 23
- 4.2 Model choice . . . 23
- 4.3 Arithmetic operations . . . 23
  - 4.3.1 Integer square root . . . 24
- 4.4 Sampling methods . . . 24
- 4.5 Result reversal . . . 25
- 4.6 Scratchpads . . . 25
  - 4.6.1 Addition . . . 26
  - 4.6.2 Multiplication . . . 26
  - 4.6.3 Square root . . . 27
- 4.7 Attention heatmaps . . . 28

5 Experimental setup 29
- 5.1 Datasets . . . 30

5.2 Evaluation . . . 30
5.3 Error analysis . . . 31
5.4 Baseline experiments . . . 31
5.5 Model size experiments . . . 31
5.6 Sampling strategy experiments . . . 32
5.7 Result reversal experiments . . . 32
5.8 Scratchpad experiments . . . 33
5.9 Attention heatmaps . . . 36

6 Experimental results **37**
6.1 Baseline experiments . . . 37
6.1.1 Error analysis . . . 40
6.2 Model size experiments . . . 44
6.3 Sampling strategy experiments . . . 45
6.4 Result reversal experiments . . . 47
6.4.1 Accuracy development . . . 48
6.4.2 Error analysis . . . 48
6.5 Scratchpad experiments . . . 49
6.5.1 Accuracy development . . . 51
6.5.2 Error analysis . . . 53
6.6 Attention heatmaps . . . 60
6.6.1 Addition task . . . 61
6.6.2 Multiplication task . . . 64
6.6.3 Square root task . . . 65

7 Summary **66**
7.1 Results . . . 66
7.2 Future work . . . 67

Bibliography **68**
References . . . 68
Preprints . . . 71
Other Links . . . 72

1 Introduction

In 2017, the influential paper "Attention Is All You Need" [1] was published. It showed that a new language model architecture, which they called the "Transformer", outperformed all previous models on English-French and English-German translation tasks, while also having lower training cost [1, p. 8]. Soon thereafter, variations of the transformer architecture described in this paper became the state of the art in most natural language processing tasks. They were also found to be competitive with established neural network architectures in other domains such as image classification [2, p. 6].

OpenAI's release of the original ChatGPT web service in late 2022 based on GPT-3.5 [42] made Transformer networks a topic of relevance for the general public. This AI chat assistant gave natural and helpful answers, mimicking human dialogue to a degree not seen before; while showing strong skills in a wide array of domains such as computer science, history and creative writing [3] [32, p. 5].

While the details of the architecture of GPT-3.5 have not been released [42] [32], it is assumed that it is based on a transformer decoder architecture very similar to that of GPT-2, which was released by OpenAI in 2019, with the main differences being that GPT-3.5 is based on a bigger model with more learnable parameters as well as a larger training dataset [43].

Even though transformer models are very strong at learning the patterns of human language, they do have limitations in other areas such as performing arithmetic. For example, GPT-3.5 and GPT-4 are unable to correctly multiply numbers with four or more decimal digits. It does however seem capable of adding numbers of arbitrary size [32] [33, p. 9]. This work seeks to explore the limits on what transformer-style models are able to learn in terms of integer arithmetic.

1.1 Natural language versus arithmetic

From a theoretical standpoint, one could argue both that natural language is harder to learn than arithmetic or vice versa: Natural language does not always follow strict logic but is more based on established conventions of its speakers, it covers a wide range of topics and can have multiple layers of meaning and nuance. Arithmetic on the other hand has strict rules with often only one correct result. Computational complexity can be high for some arithmetic tasks.

Traditionally, computers excelled at arithmetic (due to the fact that it is easy to program them to handle it) but failed at natural language. Only recently, after a few generations of transformer networks and roughly at the point in time ChatGPT was launched in late 2022, have computers managed to handle natural language reasonably well [44]. One might think that an advanced system that is competent at handling natural language would be able to handle arithmetic tasks even better. But that is not necessarily the case.

From a theoretical standpoint, Hahn [4] showed that transformer networks cannot handle some fairly basic tasks like PARITY (testing whether the number of 1s in a bitstring is even) no matter their weights or depth. This however assumes handling the language in a general sense, with unbounded string length. For inputs with limited length, they can in fact learn such languages.

In practice, OpenAI's technical report on GPT-4 [32] as well as Yang et al. [33, p. 9] showed that GPT-3.5 and GPT-4, which were among the most advanced models for natural language understanding at the time of release, struggle with multiplication of integers with four or more decimal digits. They did, however, observe good performance on adding large integers, highlighting that multiplication is not only harder from a computational complexity standpoint but also a greater challenge for transformer models.

1.2 Related work

Basic feed-forward networks can do addition of binary numbers (each input / output neuron representing one bit) with one hidden layer and binary multiplication with two hidden layers with perfect accuracy, as shown by Franco and Cannas [5]. The authors did not train a network, but instead proved its existence by constructing a feed-forward neural network including weight and bias matrices and proving its output always matches the sum or product of the input numbers.

Hoshen and Peleg [6] trained feed-forward neural networks on integer addition and multiplication, experimenting both with one-hot decimal digit vectors as well as 2D binary images of the written numbers as inputs and outputs. For addition, input integers were limited so their sum is at most seven decimal digits long. For multiplication operands were limited so the product is at most seven digits long, thus allowing operands in the range $[0, 3162]$. Error rates achieved were low for addition ($\approx 2\%$ for both for vector and image inputs), but high for multiplication ($\approx 38\%$ for vector inputs and $\approx 72\%$ for image inputs).

The main inspiration for our work was an experiment by Lee et al. [7] to teach arithmetic to small transformer networks ("nanoGPT" with about 10 million parameters). First, the authors train for 3-digit integer addition and compare final model performance when various training dataset sizes were used. This is followed by experiments using techniques like result reversal (reversing the order of output digits so the least significant digit comes first, this matches order of computing digits when adding by hand) and step-by-step thinking (not just having inputs and their sum in each training sample, but also intermediate steps of the computation, like intermediate sums and carry amounts, similar to adding or multiplying by hand) to improve performance when training with larger operands. They also experiment with 2-digit multiplication and approximating the square root of decimal numbers in the range $[0, 10]$.

A much larger transformer network with about 7 billion parameters is trained by Liu and Low [34]. The authors train on addition and multiplication up to 16 digits while using step by step thinking (including intermediate results of each computation into the training sample, similar to doing addition / multiplication by hand). Accuracy was near-perfect on addition but their model failed to learn multiplication.

Training transformer models on binary addition and multiplication was attempted by Maltoni and Ferrara [8]. This stands in contrast to the base 10 digits used in the papers mentioned before. The authors train on operands of only seven bits and use almost all possible $2^7 \cdot 2^7 = 2^{14}$ addition or multiplication examples in the training set, with the remaining few reserved for the test set. Training eventually achieved perfect accuracy for addition and multiplication, although the operand sizes were small for this study.

Length generalization is the capability of the model to work on operand lengths larger than the ones seen in training. This is the focus of McLeish et al. [9]. The authors use a large training dataset of 20 million samples of mixed operand lengths. Using some tweaks to the model's embeddings, they were able to achieve near perfect accuracy for addition and multiplication, as well as some degree of length generalization for addition.

Various transformer models from 10 million to 2 billion parameters are trained on arithmetic tasks by Yang et al. [33]. Training is divided into two phases, with the first phase containing operations with operands up to 5 digits and the second phase containing operations with operands up to 12 digits. This led to over 99% accuracy for 3-digit and 5-digit addition for all model sizes. For 3-digit multiplication, final accuracy is between 78% (10 million parameters model) and 98% (2 billion parameters model), with larger models achieving higher accuracy. For 5-digit multiplication, final accuracy is between 42% (10 million parameters model) and 90% (2 billion parameters model), with larger models again achieving higher accuracy.

The idea of training a model to perform arithmetic computations step by step (meaning the training samples also contain intermediate results similar to computing the result by hand) is explored by Deng et al. [35]. After training the model on the step by step samples, the model is trained to no longer output the intermediate steps, but only the final result. This then yields a model that simply outputs the correct result for some arithmetic task.

In addition to the input token sequence, transformer networks usually also receive an additional position vector for each token that helps them to know the position of a token in the sequence (since their learnable parameters are independent of token position). Positional encoding schemes are ways in which positional information is embedded into the token sequence to make the model grasp the order of tokens. This ranges from no positional encoding at all to more sophisticated schemes involving vector rotation. Various positional encoding schemes are compared for performance by Shen et al. [36], with a focus on learning arithmetic and length generalization.

The topic of length generalization is also explored by Cho et al. [10], in the size of operands as well as the number of operands (e.g., multiple additions or multiplications per prompt). Length generalization for the operations of addition, multiplication and modular arithmetic is also studied in [37]. Various positional encoding schemes were tested. Perfect accuracy for addition from 6 to 20 digits was achieved for all schemes. For multiplication, models where trained on operations where the smaller operand has 3 decimal digits and the larger operand has 5 decimal digits. Then, the performance of the trained models was tracked when the larger operand has no longer 5 digits, but up to 35 digits (length generalization). Multiplication with two large operands of 5 or 10 digits each was not tested.

2 Transformer models

Machine translation, which is the automated translation from one natural language to another, was an important challenge since the beginning for the field of artificial intelligence and machine learning. The input and output texts are usually modeled as sequences of integers from a limited range, with each integer representing a phrase, word, or part of a word. Classical dense/feed-forward neural networks are a poor fit for such tasks, as the sequence lengths of even a short text combined with the many possible choices for each position make for a dense network with far too many parameters.

Thus researchers started using approaches like recurrent neural networks (RNN) [11], convolutional neural networks (CNN) [12] and long short-term memory (LSTM) [13]. While these approaches did make the parameter count more manageable due to their sparse architecture, they still had issues with training on long sequences and learning long-distance correlations within a sequence.

Transformer models were introduced in the influential paper "Attention as all you need" [1] in the context of machine translation to help address the aforementioned issues. Later on, they proved useful for many other tasks.

Due to their parallelized architecture which will be explained in detail later, transformers allow for training on long sequences in a single optimization step, greatly increasing training throughput [1, p. 1]. They also have full connection between any two points in the sequence in each layer of the network, mitigating the problem of long-range dependencies being only considered across multiple model steps/layers in CNNs or RNNs which makes information loss more likely [1, p. 6].

2.1 Types of transformer models

There are a variety of different albeit similar structures that have been used to build effective transformer networks. In particular, decoder, encoder-decoder, and encoder models need to be differentiated.

Encoder models such as BERT [14] transform any given text into a vector of fixed size. This vector in the ideal case captures the essence of the text's meaning and can thus be used for tasks like text classification or sentiment analysis.

Encoder-decoder models like the original transformer architecture as described in [1] use the vector generated by the encoder to start generating new text. This can be used for example by translation models or for answering questions.

Decoder models like the GPT models [38] generate new text to extend some given starting text. This can be used for example to write a story when given a first paragraph. But with the right training, they can be used for the same tasks as encoder and encoder-decoder models as can be seen in the next section.

2.2 Transformer decoder models

The purpose of a decoder transformer model is to have it complete a sequence of tokens. These tokens may represent a letter of some alphabet, fragments of words, entire words, keywords of a programming language, even decimal digits, byte values or the binary values 0 and 1. They could even represent color values or grid coordinates. Natural language modeling, however, is the most common scenario.

The model can be trained on token sequences generated from natural language taken from books, news articles, online communication platforms or other sources. This would make the

model useful for writing stories, filling gaps in text and so on. Or it could be trained on pieces of source code, which would allow the model to act as a code completion assistant. The model would be given a piece of text that starts a story the user wants to have completed, or a piece of code that only has a function defined with comments. A well-trained model would then complete the code or story in a useful way so the rest of the story matches the theme set in the starting paragraph, or the code generated fits the comments and function signature written before.

The training texts could also all be in a structured question-answer pair form, allowing the model to answer questions. A few examples:

- Translation tasks: "en: The house is big. de: Das Haus ist groß."
- Classification tasks: "text: what a cool product, would buy again sentiment: happy"
- Mathematical tasks: "21*2=42" or "123: odd number" / "124: even number" [33, p. 3]

When the models is well-trained on these question-answer pair texts, it can be asked a question by simply leaving out the answer part at the end of the text, like "21*3=", as demonstrated in [45]. A good model would be able to generate the correct answer for similar questions by inference from the many examples it has seen, even when the exact question posed is not in its training data.

2.3 Beyond text

Apart from text-based tasks, transformer models are also capable in domains like image classification. A 2D image can be flattened into a sequence of tokens / vectors which can then be processed in a normal transformer model. The resulting image classifier achieves similar results to state-of-the-art CNN-based classifiers [2, p. 3, 5].

Transformers can even be successful at generating images from text prompts. The approach presented in [15] involves using a regular tokenizer for the text prompts and a separate discrete autoencoder to transform the training images into tokens and the generated tokens back to images.

Board games such as chess and go can also easily be tackled by transformers, as the board state and moves made can be transformed into text or represented by specialized tokenization schemes. In [16] a transformer decoder model is trained to generate best moves or score board positions based on a large dataset of human chess games. The resulting chess playing engine manages grandmaster-level performance in blitz chess against human opponents [16, p. 5].

There have been minor modifications to the model architecture, data representation, and training regime among the diverse tasks outlined above. Also the performance of the resulting models is generally good but not quite state-of-the art. Nonetheless, having one model architecture that can achieve strong performance across a diverse set of domains seems promising as a step towards artificial general intelligence.

2.4 Tokenization

Most transformer models preprocess input text using a so-called tokenizer, which is a model that was trained to group text into meaningful chunks out of a limited vocabulary called tokens to provide for an efficient representation of language [17]. The tokenizer transforms the text into a sequence of integers, which is then processed by the transformer model which outputs token IDs which are then converted back into text.

Compared to just using raw bytes as model inputs, a good tokenizer allows for shorter sequence lengths (as a token typically represents multiple characters/bytes), meaning faster generation and greater maximum length [18]. Nonetheless, there are capable models such as Google ByT5 [19] that do not need a tokenizer as they operate directly on byte values. Compared to manually specifying a vocabulary (for example contents of a dictionary) tokenization is more flexible,

having no issues with unknown words or large composite words (any possible byte sequence can still be represented) and is adaptable to the text corpus the model should be used on.

To train a tokenizer, the byte-pair-encoding (BPE) algorithm [17] is often used, which also has applications in data compression. It works as follows:

1. At the start, the set of token strings is just the possible characters in the input text, e.g., all ASCII characters. At this point, each character needs one token (string) in the representation of the tokenizer.

2. The input text is modeled as a sequence of token strings from the token strings set, preferring longer strings wherever possible over shorter ones. In this sequence, the most common adjacent pair of two token strings is identified.

3. A new token string is created from the concatenation of this pair, e.g., if "A" followed by "B" is the most common token string pair in the text, a new token string "AB" is added to the set.

4. Steps 2 and 3 are repeated over and over. This causes the token vocabulary to grow and the average number of tokens needed per character to decrease. This is done until a good balance between token vocabulary size and tokens per character is achieved.

Apart from the regular token vocabulary, most tokenizers also contain some special token values that will not be found in the input text. These are used for example to represent the start and end of each training sample, delimit individual pieces of text when they are fed to the model, or separate parts of a question/answer pair [46].

2.5 Input and output

Formally, the input to the model consists of a sequence of integers (token IDs) $x_1 \dots x_n$, where n is the length of the token sequence. Generally, transformer models place a limit on the length of the sequence n due to their internal structure, we denote this limit as n_{pos}. However, models with no such limitation have also been proposed [20]. Each integer x_i references one of the possible tokens of the model ($1 \leq x_i \leq d_{\text{vocab}}$). d_{vocab} is the size of the token vocabulary, which depends on the tokenization process used for the model [1, p. 2].

For each position in the input sequence, the model outputs a probability distribution, predicting the next token based on the input tokens up to this position. The output of the model is a sequence of probability distributions $\mathbf{y}_i$ ($1 \leq i \leq n$). $\mathbf{y}_i \in \mathbb{R}^{d_{\text{vocab}}}$, $0 \leq (\mathbf{y}_i)_j \leq 1$ [1, p. 5] [47]. $\mathbf{y}_i$ should model which token most likely follows after all the tokens $x_1 \dots x_n$.

During training, all output positions are useful as they allow to optimize the model to correctly predict x_2 based on x_1, x_3 based on x_1 and x_2, x_n based on $x_1 \dots x_{n-1}$, all in a single training step. This greatly reduces training time for longer sequences [48].

2.6 Text generation

When using the trained model to generate text, only the last output distribution, $\mathbf{y}_n$, is of interest, because it tells the next token after the given input sequence. [49]

This is used to generate one new token at a time (autoregressive decoding). The new token is picked from the distribution either by choosing the most likely token (greedy decoding), random sampling according to the probability distribution, or more sophisticated methods such as top-p sampling (choosing among the most likely tokens whose probability mass equals for example $p = 0.95$) and top-k sampling (selecting from the for example $k = 10$ most likely tokens) [21].

The newly generated token x_{n+1} is appended to the input sequence and the whole process is repeated, with n growing by one after each step, until a special end-of-sequence token is generated, the maximum length n_{pos} is reached, or some other stop condition is triggered [47,

sect. "Generation"]. If generation is not terminated, the model could in theory generate a text of infinite length, although transformer models generally only have a limited window of past tokens they can consider when generating the next token. In addition, generation time and memory requirements generally increase quadratically with sequence length for transformer models (section 3.8.4).

2.7 User interaction

In 2023, OpenAI released their most successful product yet, ChatGPT [42]. This online service allows users to chat with an artificial intelligence assistant which can help with reciting facts, writing texts and code, giving advice of all sorts and so on.

The principle by which transformer decoder models can act as chat assistants is simple: The model is trained on conversations like this:

```
User: What color is the sky?
Assistant: The sky is blue.
```

The model can then become the helpful assistant by filling in the assistant's messages:

```
User: What color is a banana?
Assistant:
```

and the model would generate a response like "A banana is yellow" followed by an end-of-sequence token, similar to its training examples. For more details on how generic language models can be tuned to follow instructions or conversation, see [22] and [23].

2.8 Generative pre-training

The GPT-2/GPT-3/ChatGPT transformer models have demonstrated the effectiveness of a technique used to train language models called generative pre-training (GPT). This technique was first popularized in [38] and consists of two phases.

In the pre-training stage the model is first trained on a large corpus of text in multiple languages, including websites, books, articles, chats and source code. In this manner, a huge amount of training data can be collected without the need for careful manual selection / preparation of question-answer pairs. The training data is split into appropriate chunks and the model learns to predict (generate) the next word based on the previous ones for each chunk as a form of unsupervised learning [38, p. 3].

After sufficient pre-training, the model is then fine-tuned on question-answer pairs of the specific task the model should be able to solve, like the examples presented in section 2.2 or even human-assistant chats on a wide range of topics.

Due to the well-known phenomenon of transfer learning [24], the pre-trained model will have already internalized the structure of natural language, source code and math. This allows a relatively small amount of hand picked fine-tuning examples to make it significantly stronger compared to just training on the question-answer pairs on a freshly initialized model [38, p. 8].

2.9 Attention mechanism

Before transformers, model architectures such as recurrent neural networks (RNN) and long short-term memory (LSTM) were commonly used to handle textual or other sequential data. They ingest the tokenized text one token at a time, keeping one or multiple fixed vectors of model state. Transformers, in contrast, dropped recurrence and handle the entire sequence in parallel [1, p. 1].

To draw connections between elements of the token sequence, they use a so-called attention mechanism. It is in fact the only part of the model that connects the different parts of the sequence it is processing, all other components of a transformer model handle each position independently. The concept of an attention mechanism is best explained using a high-level example [48]: Suppose the model is processing the sentence "A robot must obey orders given to it" and each word is one token. The model should learn that there is a connection from the pronoun "it" to the earlier noun "robot". The model's attention mechanism would do the following when handling the token "it":

1. Generate a query vector $\mathbf{q}_{\text{it}} = f_q(\text{"it"})$ for the token "it" using a function f_q that depends on learned parameters of the model (often a linear transformation).

2. Generate key vectors $\mathbf{k}_t$ for all the tokens t of the sentence, using another learned function f_k: $\mathbf{k}_{\text{a}} = f_k(\text{"a"})$, $\mathbf{k}_{\text{robot}} = f_k(\text{"robot"})$, $\ldots$, $\mathbf{k}_{\text{it}} = f_k(\text{"it"})$.

3. Compute numeric similarity scores s between the query vector and each key vector, using a measure like dot product: $s_{\text{a}} = \mathbf{q}_{\text{it}} \cdot \mathbf{k}_{\text{a}}$, $s_{\text{robot}} = \mathbf{q}_{\text{it}} \cdot \mathbf{k}_{\text{robot}}$, $\ldots$, $s_{\text{it}} = \mathbf{q}_{\text{it}} \cdot \mathbf{k}_{\text{it}}$.

4. Normalize the similarity scores s using softmax to get the normalized scores s'_t for each token t. This ensures that $\sum_t s'_t = 1$.

5. Generate value vectors $\mathbf{v}_t$ for all the tokens t of the sentence, using another learned function f_v: $\mathbf{v}_{\text{a}} = f_v(\text{"a"})$, $\mathbf{v}_{\text{robot}} = f_v(\text{"robot"})$, $\ldots$, $\mathbf{v}_{\text{it}} = f_v(\text{"it"})$.

6. Using the normalized similarity scores, blend the value vectors together to get the attention output vector for "it": $\mathbf{a}_{\text{it}} = \sum_t s'_t \mathbf{v}_t$.

Assuming the model has learned to generate meaningful query and key vectors, the similarity score to the query of "it" should be much higher for the key of "robot" than for the other positions' keys. This would cause the attention output vector of "it" to mostly consist of the value vector of the token "robot", with the other positions' value vectors having little impact. Thus data flows to the position of the token "it" mostly from the position of the token "robot", which models the linguistic connection between these words.

It has to be said however that this explanation has been simplified considerably. Transformer models do not operate on words or tokens directly but first transform them into vectors of fixed size that are then fed through the attention mechanism. There are multiple stacked blocks inside a typical transformer model, each consisting of an attention mechanism among other components. A more precise description of the attention mechanism of a realistic transformer model is given in section 3.8.4.

3 OpenAI GPT-2

A good example of a successful decoder model is OpenAI's GPT-2 [39] [50] [47, source code]. It was released in 2019 and demonstrated the ability to write realistic articles based on a given starting prompt. This model is useful for explaining the details of the transformer architecture, because the model weights and code were released, its proven effectiveness in generating meaningful text, and due to the fact that its even more powerful successors have not experienced significant architectural changes [43].

3.1 Achitecture

A general overview of the architecture of GPT-2 is presented in fig. 3.1. As described in section 2.5, the input to the model is an integer sequence, with each integer representing an element of the token vocabulary. Creating this token sequence from the input text is done by the tokenizer (section 2.4, section 3.3). This tokenizer is considered a component separate from the model and is trained before it. The model is then trained based on the output indices of the chosen tokenizer, therefore the tokenizer cannot be changed for a trained model. The various stages inside the model are as follows:

1. The sequence of token indices is then transformed into a sequence of vectors by the preprocessing stage (section 3.4). The preprocessing stage converts each token $x_i \in \mathbb{N}$ into a hidden vector $\mathbf{h}^{(0)}{}_i \in \mathbb{R}^{d_{\text{hidden}}}$ based on the token value x_i and token position i. For this, it uses learnable embedding matrices. The size of each of these hidden vectors, d_{hidden} is a parameter that can be configured when creating a GPT-2 model (section 3.2).

2. These vectors $\mathbf{h}^{(0)}{}_i$ are then fed through a number of GPT-2 blocks, each being identical except for having its own set of learnable parameters. These GPT-2 blocks are indexed by l ($1 \leq l \leq n_{\text{blocks}}$), with n_{blocks} being the number of GPT-2 blocks the model has, which can also be configured at model creation. We denote the input vectors to block number l as $\mathbf{h}^{(l-1)}{}_i$ and the output vectors of that block as $\mathbf{h}^{(l)}{}_i$. Thus for a model with, e.g., 12 blocks, the inputs to the first block are $\mathbf{h}^{(0)}{}_i$ and the outputs after the last block are $\mathbf{h}^{(12)}{}_i$.

   Each block outputs a vector sequence of the same dimensions as its input. The GPT-2 block structure is described in detail in section 3.8 and consists of multiple layers including normalization, a feed-forward network and the attention mechanism. The attention mechanism (section 3.8.4) is special to transformer networks and the only point in the model where vectors are not transformed independently, i.e., the output vector of the attention mechanism at position i depends not just on the input vector at position i, but also on the vectors at all previous positions $1 \leq i' < i$. The job of the block and the attention mechanism in particular is to allow the model to learn how later elements of the training sequences are influenced by previous elements, e.g., capitalized words mostly appearing after points and spaces (ends of sentences) in English training texts.

3. After the last block there is the postprocessing stage (section 3.5). Its job is to convert each hidden vector $\mathbf{h}^{(n_{\text{blocks}})}{}_i$ into a probability distribution $\mathbf{y}_i \in \mathbb{R}^{d_{\text{vocab}}}$. This is also done with a learnable embedding matrix.

   During training (section 3.7), the model parameters are optimized so that every $\mathbf{y}_i$ correctly predicts the next token in the training sequence, x_{i+1}. During inference (section 3.6), only the last distribution, $\mathbf{y}_n$, is used to sample a predicted token x_{n+1}, which then is appended to the input sequence for the next generation step.

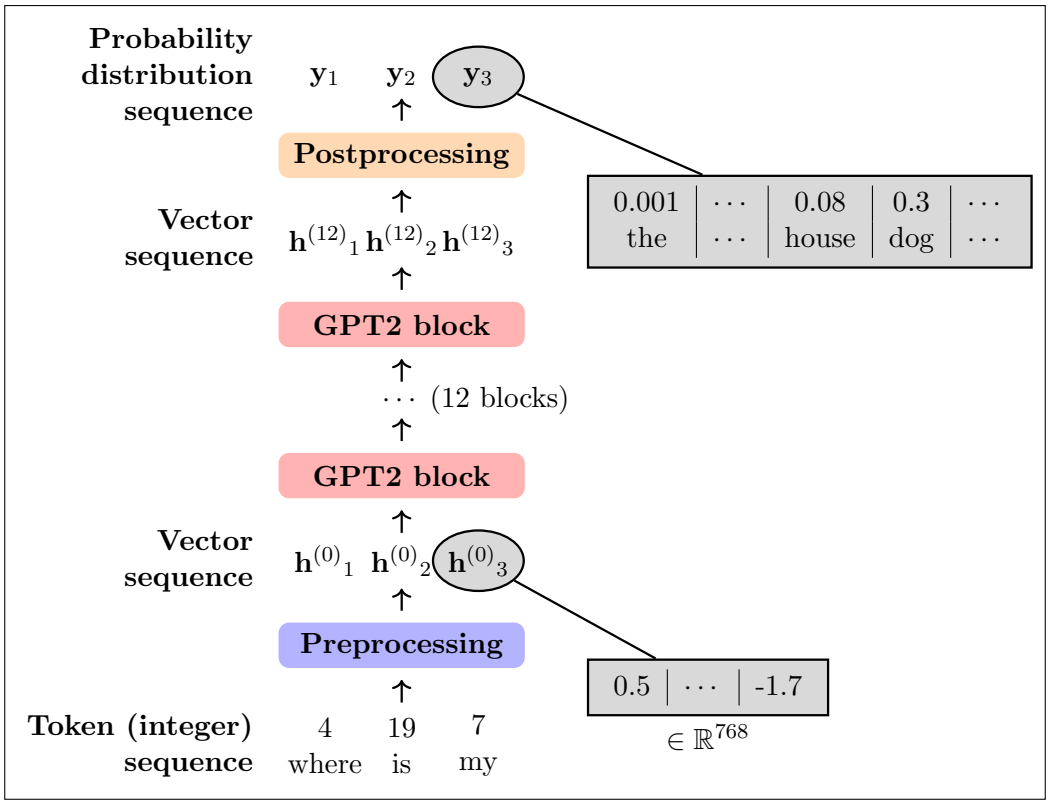

Figure 3.1: Overview of a GPT-2 model. This example uses a model with $n_{\text{blocks}} = 12$ blocks, $d_{\text{hidden}} = 768$ hidden vector size. The model is processing a sequence of length 3. Models can also process longer sequences than shown in this example, up to their positional limit n_{pos} which is configurable at model creation time as is the number of blocks and hidden vector size.

3.2 Model size variants

To make the model bigger (needing more memory and computation time, but allowing for more accurate generation) or smaller (faster, but lower quality output), two hyperparameters can be adjusted [39, p. 4] [38, p. 5]: The vector size for each position can be changed just like the number of transformer blocks. The number of attention heads is usually adapted so that the size of each head remains around 64 [39, p. 4] [51]. OpenAI GPT-2 was released in four versions with different parameter counts, as seen in table 3.1:

Model variant	No. of parameters	n_{blocks}	d_{hidden}	n_{heads}	d_{head}
gpt2-small	117M	12	768	12	64
gpt2-medium	345M	24	1024	16	64
gpt2-large	774M	36	1280	20	64
gpt2-xl	1558M	48	1600	25	64

Table 3.1: Versions of GPT-2 released with different parameter counts [51]: n_{blocks} is the number of GPT-2 blocks, d_{hidden} is the vector size at each position, n_{heads} is the number of attention heads and d_{head} is the size of each attention head.

3.3 Tokenization

The GPT-2 models as released by OpenAI were trained using a Byte-pair encoding (BPE) tokenizer (section 2.4) with a vocabulary of 50257 tokens [39, p. 4] [47]. The tokenizer was

itself trained on a natural language corpus separately before training the actual models. The models were then configured with $d_{\text{vocab}} = 50257$ and the training texts were processed using this tokenizer before being fed into the model. Thus, when using these models with the weights as released by OpenAI, the same tokenizer must be used for text generation during inference time as well. An example on how the tokenizer that was released with OpenAI GPT-2 handles a sentence can be seen in fig. 3.2.

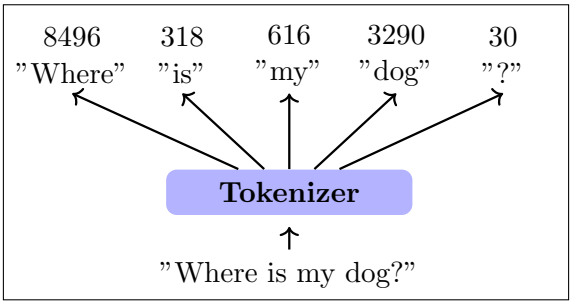

Figure 3.2: Operation of the GPT-2 tokenizer on an example sentence.

It is also possible to train a GPT-2 model with a different tokenizer with, e.g., a smaller vocabulary. Instead of a proper tokenizer that was trained to efficiently chunk text in some natural language, other GPT-2 implementations like nanoGPT [52] use a simpler approach: Each ASCII character in the training text is mapped to one token, with the token value equal to the ASCII character value. This makes for a simpler overall implementation that keeps all the processing outside the tokenizer and inside the actual model. The downside is that token sequences get significantly longer, thus a model with the same n_{pos} and ASCII tokenization can only handle significantly shorter texts than one with a trained tokenizer that recognizes, e.g., frequent words and groups all their characters into a single token.

3.4 Preprocessing

Transformer networks do not operate on integer token IDs, but on state vectors from $\mathbb{R}^{d_{\text{hidden}}}$. d_{hidden}, the dimension of the state vector at each position, is a parameter fixed at the time of model construction (section 3.2). This means the token sequence needs to be converted into a sequence of such vectors before feeding it through the model. This happens during the preprocessing stage, which is illustrated in fig. 3.3. For converting the token IDs x_i into initial state vectors $\mathbf{h}^{(0)}{}_i$, GPT-2 has a learnable token embedding matrix $\mathbf{E}_{\text{tok}} \in \mathbb{R}^{d_{\text{vocab}} \times d_{\text{hidden}}}$. Each row of this matrix stores the learnable initial state vector for the corresponding token ID.

Because the blocks of the GPT-2 have no positional awareness (their learnable parameters do not depend on the sequence position i), information about the position of a token within the sequence needs to be included as well [1, p. 6]. In GPT-2, this is done by a learnable embedding matrix $\mathbf{E}_{\text{pos}} \in \mathbb{R}^{d_{\text{vocab}} \times d_{\text{hidden}}}$ storing a learnable vector for each position up to the sequence length limit n_{pos}.

The starting vector $\mathbf{h}^{(0)}{}_i \in \mathbb{R}^{d_{\text{hidden}}}$ for a token value x_i at position i is then given in eq. (3.1) as the sum of the token and positional embedding vectors [1, p. 3]:

$$\mathbf{h}^{(0)}{}_i = \mathbf{E}_{\text{tok}}[x_i, :] + \mathbf{E}_{\text{pos}}[i, :] \tag{3.1}$$

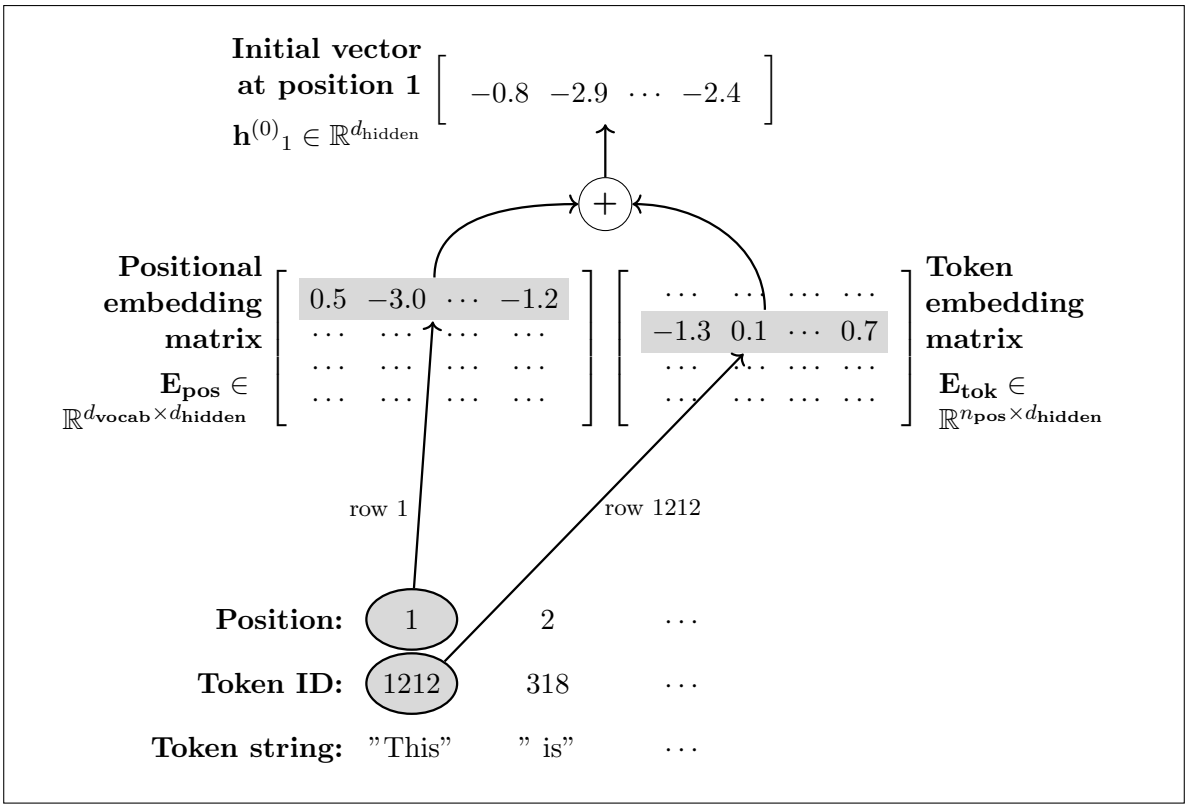

Figure 3.3: GPT-2 preprocessing stage on an example token sequence. Based on the token position and value, embedding vectors are summed to give the starting vector at this position.

3.5 Postprocessing

GPT-2 uses a "language modeling head" [47] to convert the outputs of the last block $\mathbf{h}^{(n_{\text{blocks}})}{}_i$ into token probabilities $\mathbf{y}_i$, the steps of which are illustrated in fig. 3.4.

First, a final layer normalization step is performed [39, p. 4], which transforms $\mathbf{h}^{(n_{\text{blocks}})}{}_i$ into $\mathbf{h}^{(n_{\text{blocks}},norm)}{}_i$. The details of this layer normalization step are the same as the layer normalization steps in each GPT-2 block, which are described in section 3.8.2.

GPT-2 then re-uses the same matrix $\mathbf{E}_{\text{tok}}$ that was used in the preprocessing stage to convert $\mathbf{h}^{(n_{\text{blocks}},norm)}{}_i$ into a token probability vector $\mathbf{y}'_i$ of dimension d_{vocab}, using matrix-vector product, as shown in eq. (3.2) and eq. (3.3). This tying of the weights in the postprocessing stage to those of the preprocessing stage is done for improving model performance as well as reducing the number of learnable parameters [25].

Finally, softmax is applied to transform this vector $\mathbf{y}'_i$ into a proper probability distribution $\mathbf{y}_i$ (eq. (3.4)).

$$\mathbf{y}'_i = \mathbf{E}_{\text{pos}} \mathbf{h}^{(n_{\text{blocks}},\text{norm})}{}_i \tag{3.2}$$

$$(\mathbf{y}'_i)_k = \sum_{j=1}^{d_{\text{hidden}}} \mathbf{E}_{\text{pos}}[k,j](\mathbf{h}^{(n_{\text{blocks}},\text{norm})}{}_i)_j \qquad 1 \leq k \leq d_{\text{vocab}} \tag{3.3}$$

$$\mathbf{y}_i = \text{softmax}(\mathbf{y}'_i) \tag{3.4}$$

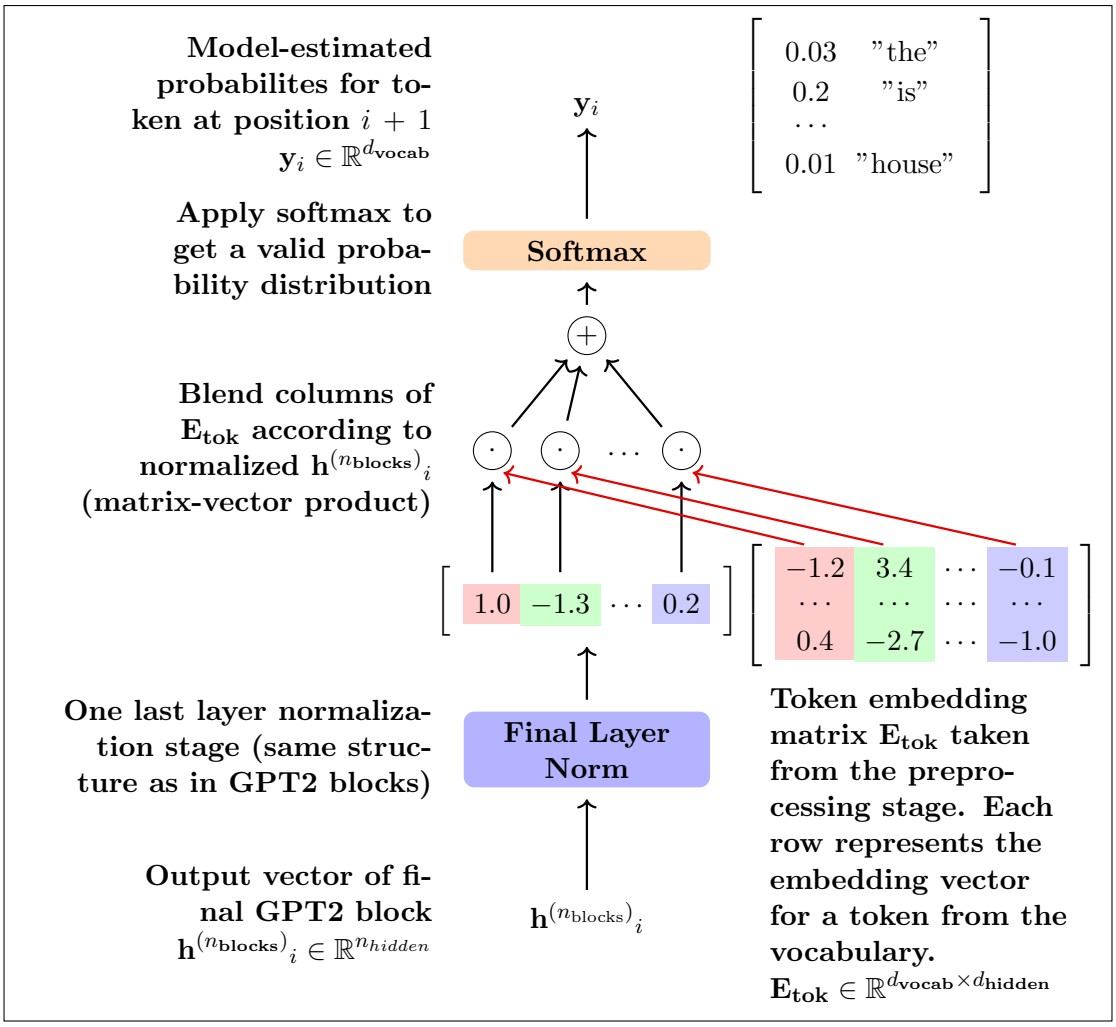

Figure 3.4: Detailed example of the GPT-2 postprocessing stage. $\mathbf{h}^{(n_{\text{blocks}})}{}_i$, the output of the last GPT-2 block, is converted into token probabilities.

3.6 Text generation

Generating output text using a trained GPT-2 model follows the general process for transformer networks outlined in section 2.6. The generation process is visualized in fig. 3.5. It always starts with some existing starting text that is transformed into a sequence of n tokens $x_1 \dots x_n$. These tokens are fed through the model to get the output probability distributions $\mathbf{y}_1 \dots \mathbf{y}_n$. The last distribution $\mathbf{y}_n$ is sampled using one of the sampling methods outlined in section 2.6. The token that results from this sampling becomes x_{n+1} and is appended to the sequence $x_1 \dots x_n$. The process outlined above is then repeated, now sampling $\mathbf{y}_{n+1}$ to get the next token x_{n+2} and so on.

Depending on what texts / token sequences the model was trained on, it will at some point cause a special stop token ID to be sampled, which tells the sampling process that the generated text is now complete and sampling should be stopped. In other cases, a fixed number of tokens defined by the user might be sampled, which is possible as long as the total token sequence length does not exceed n_{pos}.

Due to the limited dimensions of the positional embedding matrix $\mathbf{E}_{\text{pos}}$, GPT-2 as described here does not support generating texts of unlimited length. Other variants of GPT-2 or similar transformer models use different positional embeddings not based on a matrix of fixed dimension. Those models could support unlimited generation, although it is limited in practice by computational complexity and limited coherence in the generated text (section 2.6).

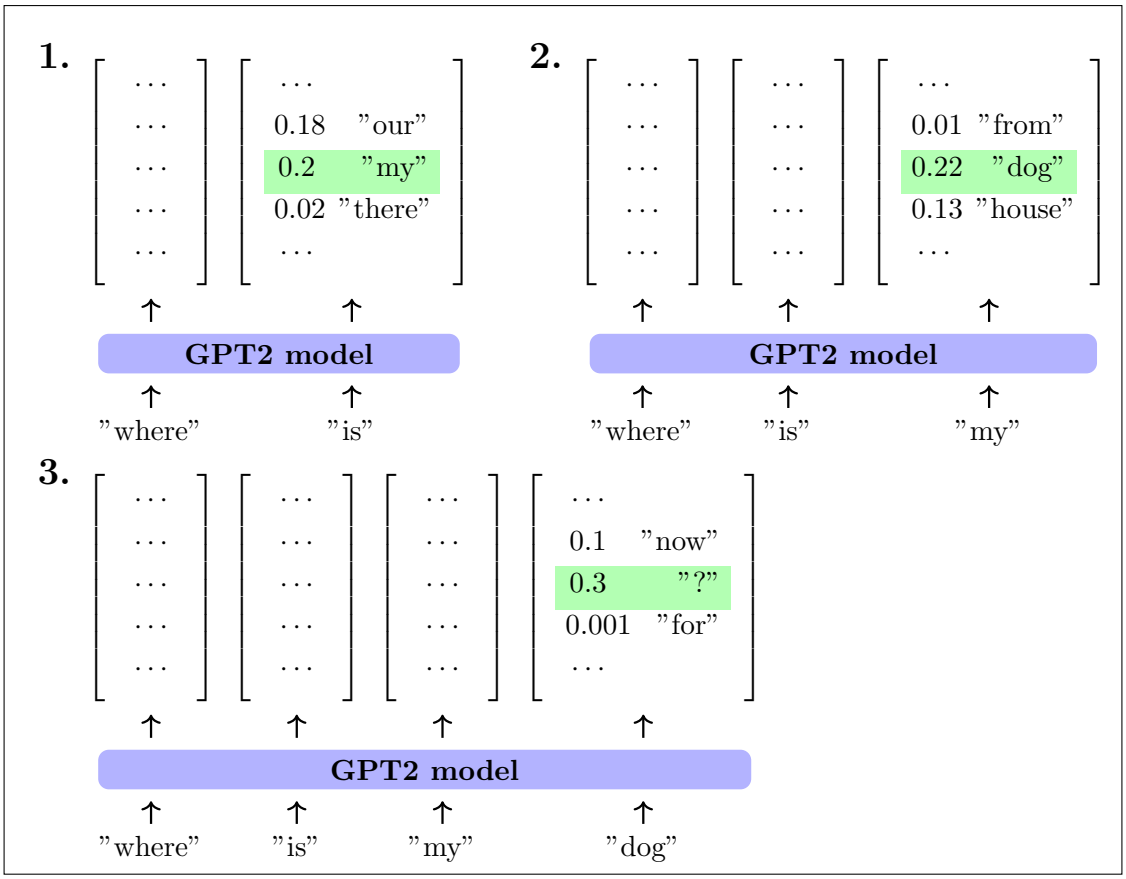

Figure 3.5: Text generation using the GPT-2 model for the example starting text "where is". The rightmost probability distribution is used to pick the next token, after which the data, augmented with this new token, is fed through the model again.

3.7 Training

GPT-2 uses a training objective similar to [38, p. 3] where a large text corpus is used to make the model predict the next token based on the tokens up to this point. In [38] this is called "unsupervised pre-training", however other authors refer to similar automated labeling techniques as "self-supervised learning" [40] [26]. In contrast to classical supervised learning, the training samples in self-supervised learning are not manually labeled, but the inputs and labels are automatically created from each training sample using some mathematical function. An example of this could be erasing some pixels of an image in the input while leaving them untouched in the label. For training transformer decoder models like GPT-2, the training goal typically is to make the model correctly complete a text similar to the training samples when only the start of such a text is given. To achieve this, each training sample's token sequence $x_1 \dots x_n$ is shifted by one position: the inputs to the model are $x_1 \dots\ x_{n-1}$ and the corresponding labels are $x_2 \dots\ x_n$. This makes the model learn to predict x_{i+1} based on $x_1 \dots\ x_i$ for all i from 1 to $n-1$ in a single optimization step (also see section 2.5).

For each probability distribution $\mathbf{y}_i$ output by the model, cross-entropy loss is used to measure the deviation between $\mathbf{y}_i$ and the ideal $\hat{\mathbf{y}}_i$, where $\hat{\mathbf{y}}_i$ is the probability distribution of a guaranteed token with ID x_{i+1} [39, p. 2] [47].

Standard initialization techniques (for example uniform initialization) can be used for the weight and bias matrices [38, p. 5].

GPT-2 is trained using the Adaptive Moment Optimizer (Adam) [38, p. 5]. A naive stochastic gradient optimizer will not achieve good results when training transformer networks like GPT-2, thus more advanced optimizers like Adam need to be used to make the model converge to a

degree where it becomes useful [27].

3.8 GPT-2 block structure

The GPT-2 blocks are the core of the model, where it learns to draw connections between the vectors at different positions, and data can flow between positions. Because its structure is somewhat complex, we dedicated this separate section to it. A high-level overview of the structure of a GPT-2 block is presented in fig. 3.6 as well as equations (3.5) to (3.12):

1. Equation (3.5) shows the input vectors to the block with index l, given as $\mathbf{h}^{(l,0)}{}_1 \dots \mathbf{h}^{(l,0)}{}_n$, are the are the outputs of the previous block $\mathbf{h}^{(l-1)}{}_1 \dots \mathbf{h}^{(l-1)}{}_n$. For the first block, its inputs are the outputs of the preprocessing stage (section 3.4).

2. The first step of each block is a layer normalization stage, described by eq. (3.6). It is explained in more detail in section 3.8.2.

3. After the layer normalization stage the vectors pass through the heart of the block - the multi-head attention mechanism. Equation (3.7) shows that multi-head attention, in contrast to the other stages, needs access to the entire vector sequence $\mathbf{h}^{(l,1)}{}_1 \dots \mathbf{h}^{(l,1)}{}_n$ as well as the position i for which to compute the attention output. This is necessary because it connects the vectors at different positions and lets data flow between them. Multi-head attention is explained in detail in section 3.8.4.

4. After the attention mechanism, there is a residual connection described by eq. (3.8), which lets data flow around the layer norm and attention stages as well as through it. The motivation behind this is explained in section 3.8.3.

5. Then, there is a second layer normalization stage described by eq. (3.9). It works the same as the first layer normalization stage, but with separate learnable parameters. It is followed by the multi-layer perceptron (MLP) stage, described by eq. (3.10). This stage is a basic feed-forward network with one hidden layer, explained in in section 3.8.6.

6. Finally, there is a second residual connection, described by eq. (3.11), whose outputs as defined by eq. (3.12) are the outputs of the block, $\mathbf{h}^{(l)}{}_1 \dots \mathbf{h}^{(l)}{}_n$.

The layer normalization, attention and MLP stages all have learnable parameters, with each GPT-2 block having a separate set of such learnable parameters. All stages except the attention stage operate on each vector $\mathbf{h}_i$ independently.

$$\mathbf{h}^{(l,0)}{}_i = \mathbf{h}^{(l-1)}{}_i \tag{3.5}$$

$$\mathbf{h}^{(l,1)}{}_i = \mathrm{LN1}^{(l)}(\mathbf{h}^{(l,0)}{}_i) \tag{3.6}$$

$$\mathbf{h}^{(l,2)}{}_i = \mathrm{MHA}^{(l)}(\mathbf{h}^{(l,1)}{}_1 \dots \mathbf{h}^{(l,1)}{}_i) \tag{3.7}$$

$$\mathbf{h}^{(l,3)}{}_i = \mathbf{h}^{(l,0)}{}_i + \mathbf{h}^{(l,2)}{}_i \tag{3.8}$$

$$\mathbf{h}^{(l,4)}{}_i = \mathrm{LN2}^{(l)}(\mathbf{h}^{(l,3)}{}_i) \tag{3.9}$$

$$\mathbf{h}^{(l,5)}{}_i = \mathrm{MLP}^{(l)}(\mathbf{h}^{(l,4)}{}_i) \tag{3.10}$$

$$\mathbf{h}^{(l,6)}{}_i = \mathbf{h}^{(l,3)}{}_i + \mathbf{h}^{(l,5)}{}_i \tag{3.11}$$

$$\mathbf{h}^{(l)}{}_i = \mathbf{h}^{(l,6)}{}_i \tag{3.12}$$

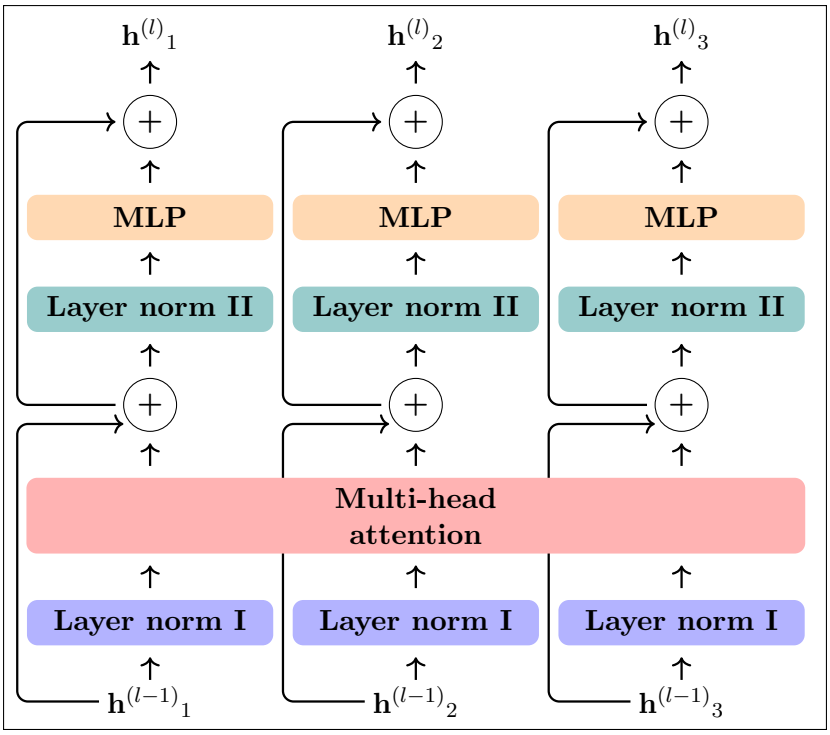

Figure 3.6: Overview of the components of a GPT-2 block with index l. This example shows three positions $\mathbf{h}^{(l)}{}_1 \ldots \mathbf{h}^{(l)}{}_3$, but the actual model is flexible regarding sequence length.

3.8.1 Notation used

To keep the diagrams and equations readable when describing the stages of the GPT-2 block (layer norm, attention, MLP) next, we define the inputs to the stage simply as $\mathbf{h}_i$ for each position i from 1 to n, intermediate results as $\mathbf{h}^{[1]}{}_i$ and the outputs as $\mathbf{h}'_i$, dropping the block index l and stage index. Each block and each stage of the block has separate, independent learnable parameters despite this not being reflected in the notation used below.

3.8.2 Layer normalization

The two layer normalization stages of the GPT-2 block are illustrated fig. 3.7 as well as equations (3.13) to (3.15).

First, the input vector $\mathbf{h}_i$ is normalized in eq. (3.13) by subtracting its mean μ and dividing by its standard deviation σ, with ϵ being a small positive constant added for numerical stability of the square root.

Then, a gain and bias value is applied per component to the intermediate result $\mathbf{h}^{[1]}{}_i$. The first layer normalization stage LN1 uses learnable gain and bias vectors $\mathbf{w}_{\text{LN1}}, \mathbf{b}_{\text{LN1}} \in \mathbb{R}^{d_{\text{hidden}}}$ as described in eq. (3.14). The second layer normalization stage LN2 uses separate gain and bias vectors $\mathbf{w}_{\text{LN2}}, \mathbf{b}_{\text{LN2}} \in \mathbb{R}^{d_{\text{hidden}}}$ as described in eq. (3.15).

$$(\mathbf{h}^{[1]}{}_i)_j = \frac{(\mathbf{h}_i)_j - \mu(\mathbf{h}_i)}{\sqrt{\sigma^2(\mathbf{h}_i) + \epsilon}} \tag{3.13}$$

$$(\mathbf{h}'_i)_j = (\mathbf{w}_{\text{LN1}})_j (\mathbf{h}^{[1]}{}_i)_j + (\mathbf{b}_{\text{LN1}})_j \tag{3.14}$$

$$(\mathbf{h}'_i)_j = (\mathbf{w}_{\text{LN2}})_j (\mathbf{h}^{[1]}{}_i)_j + (\mathbf{b}_{\text{LN2}})_j \tag{3.15}$$

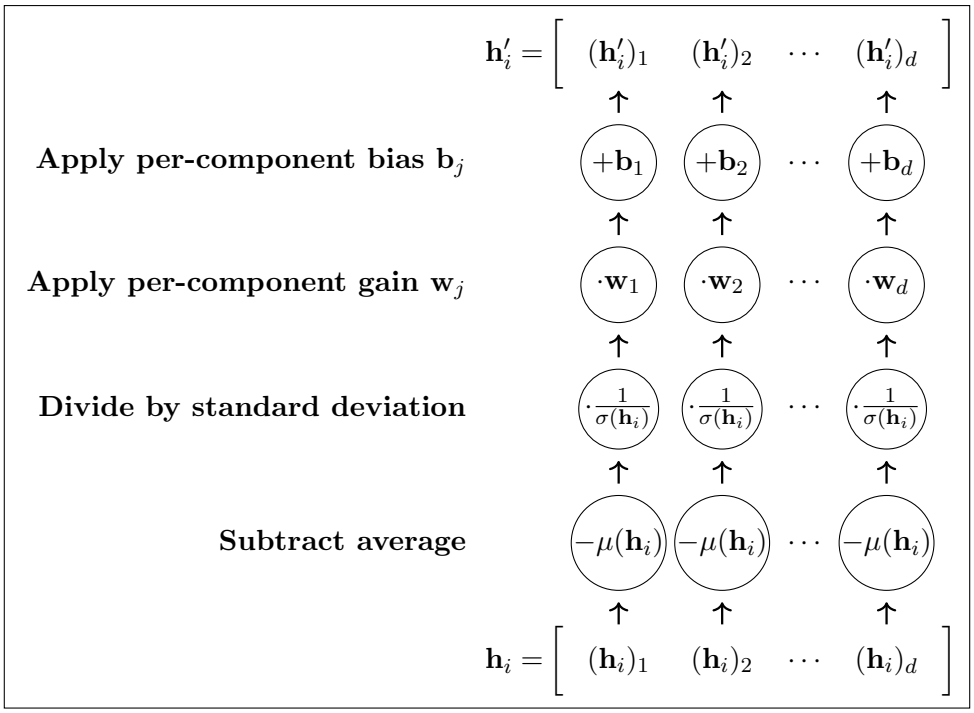

Figure 3.7: Detailed view of the layer normalization stages of the GPT-2 block. $d = d_{\text{hidden}}$. For the first layer norm LN1, $\mathbf{w} = \mathbf{w}_{\text{LN1}}, \mathbf{b} = \mathbf{b}_{\text{LN1}}$. For the second layer norm LN2, $\mathbf{w} = \mathbf{w}_{\text{LN2}}, \mathbf{b} = \mathbf{b}_{\text{LN2}}$.

3.8.3 Residual connection

A residual connection, as described in eq. (3.16), works by adding the inputs of some function f, like a layer in a machine learning model, back to its outputs [1, p. 3] [28]. When using residual connections, the input data does not need to flow through all computation steps before reaching a given point, instead it can bypass some of them, allowing later computations easier access to earlier data. Such connections are found in almost all models having many steps/layers, because they were found to greatly help when training deep models [28, p. 1-2].

$$\mathbf{h}'_i = \mathbf{h}_i + f(\mathbf{h}_i) \tag{3.16}$$

3.8.4 Multi-head attention

GPT-2, just like the original transformer introduced in [1], uses a form of attention mechanism called multi-head attention [39, p. 4].
The stages of multi-head attention are presented in fig. 3.8 and equations (3.17) to (3.25). At first, query, key and value vectors are computed for each position, which are then split into segments of equal size called attention heads. Then, the actual attention values are computed for each head, which are then merged and finally subjected to a linear projection [1, p. 4].

The reasoning behind introducing multiple attention heads is given in [1, p. 5] as allowing the model to better attend to multiple separate positions i' from some position i, each position being highlighted by a separate attention head. When just using a single attention head, blending the value vectors (see section 3.8.5) causes this information to become more diluted compared to having each i' focused in a separate attention head's output and then concatenated.

Based on the input vectors $\mathbf{h}_1 \dots \mathbf{h}_i$, the multi-head attention output $\mathbf{h}'_i$ is computed as follows:

1. For each position $i' \leq i$, a query vector $\mathbf{q}_{i'}$, a key vector $\mathbf{k}_{i'}$, and a value vector $\mathbf{v}_{i'}$ are generated using linear transforms with learnable parameters $(\mathbf{W}_q, \mathbf{b}_q), (\mathbf{W}_k, \mathbf{b}_k), (\mathbf{W}_v, \mathbf{b}_v)$, as described by equations (3.17) to (3.19). Each of these vectors has dimension d_{hidden}, just like $\mathbf{h}_i$.

2. These vectors are each split into n_{heads} segments of equal size d_{head}. This splitting including the slicing indices j_1, j_2 is described by equations (3.20) to (3.22). Each slice index h represents an attention head.

3. For each attention head h, scaled dot-product attention (section 3.8.5) is used to compute an attention output $\mathbf{a}_{i_h}$ at position i for that head, as described by eq. (3.23). This needs the query, key, and value vectors for that head at each position up to i.

4. These attention outputs $\mathbf{a}_{i_h}$ are concatenated for all heads, giving a total attention output $\mathbf{a}_i$, as described by eq. (3.24).

5. The concatenated attention outputs are passed through a linear transform with learnable parameters $(\mathbf{W}_{\text{proj}}, \mathbf{v}_{\text{proj}})$, as described by eq. (3.25). This results in the output of the multi-head attention mechanism, $\mathbf{h}'_i$.

$$\mathbf{q}_{i'} = \mathbf{W}_q \mathbf{h}_{i'} + \mathbf{b}_q \quad 1 \leq i' \leq i \tag{3.17}$$
$$\mathbf{k}_{i'} = \mathbf{W}_k \mathbf{h}_{i'} + \mathbf{b}_k \quad 1 \leq i' \leq i \tag{3.18}$$
$$\mathbf{v}_{i'} = \mathbf{W}_v \mathbf{h}_{i'} + \mathbf{b}_v \quad 1 \leq i' \leq i \tag{3.19}$$
$$\mathbf{q}_{i_h} = (\mathbf{q}_i)_{j_1:j_2} \quad 1 \leq h \leq n_{\text{heads}}, j_1 = (h-1) \cdot d_{\text{head}} + 1, j_2 = h \cdot d_{\text{head}} \tag{3.20}$$
$$\mathbf{k}_{i_h} = (\mathbf{k}_i)_{j_1:j_2} \quad 1 \leq h \leq n_{\text{heads}}, j_1 = (h-1) \cdot d_{\text{head}} + 1, j_2 = h \cdot d_{\text{head}} \tag{3.21}$$
$$\mathbf{v}_{i_h} = (\mathbf{v}_i)_{j_1:j_2} \quad 1 \leq h \leq n_{\text{heads}}, j_1 = (h-1) \cdot d_{\text{head}} + 1, j_2 = h \cdot d_{\text{head}} \tag{3.22}$$
$$\mathbf{a}_{i_h} = \text{SDPA}(\mathbf{q}_{1_h} \ldots \mathbf{q}_{i_h}, \mathbf{k}_{1_h} \ldots \mathbf{k}_{i_h}, \mathbf{v}_{1_h} \ldots \mathbf{v}_{i_h}) \tag{3.23}$$
$$\mathbf{a}_i = \text{concat}_{1 \leq h \leq n_{\text{heads}}} \ \mathbf{a}_{i_h} \tag{3.24}$$
$$\mathbf{h}'_i = \mathbf{W}_{\text{proj}} \mathbf{a}_i + \mathbf{b}_{\text{proj}} \tag{3.25}$$

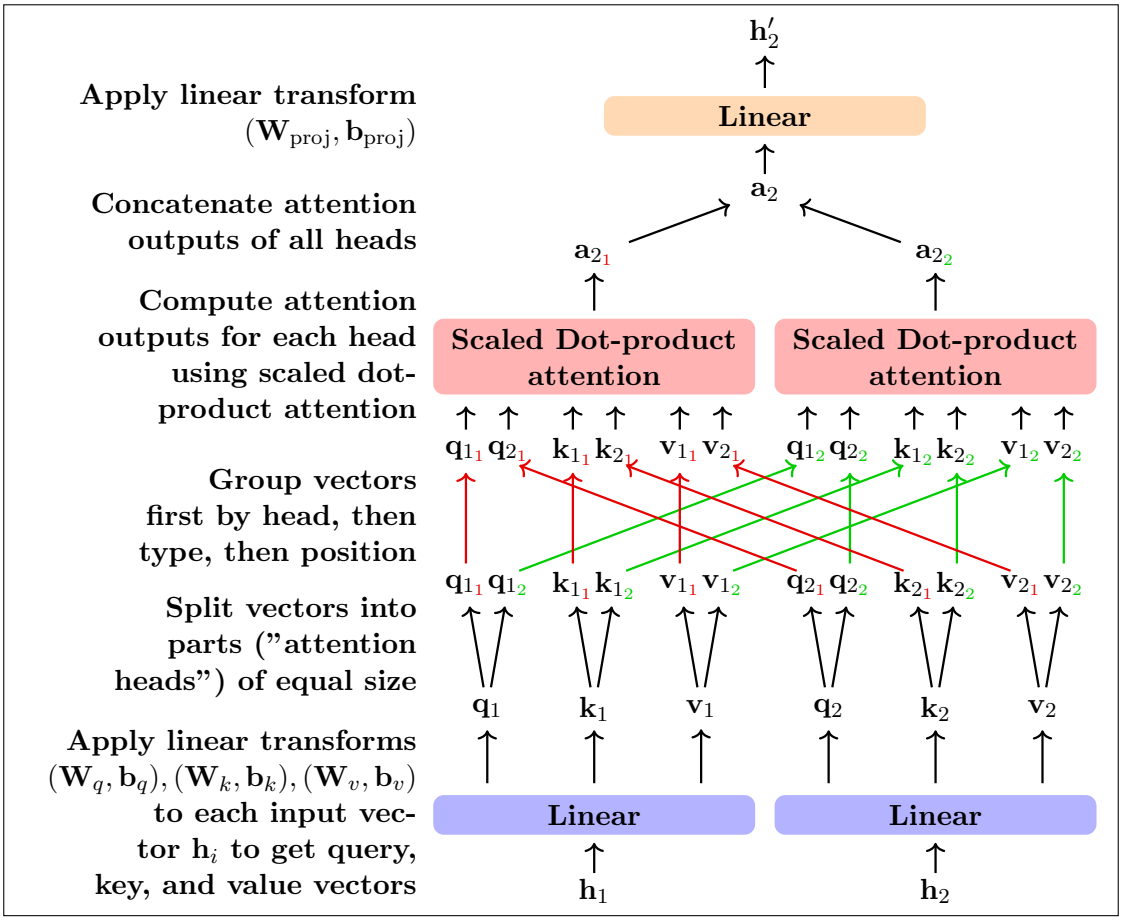

Figure 3.8: Structure of the multi-head attention mechanism. This example uses two positions $\mathbf{h}_1, \mathbf{h}_2$ and two attention heads. The actual model can in theory handle arbitrary sequence lengths and also has more attention heads.

3.8.5 Scaled dot-product attention

The scaled dot-product attention mechanism as outlined in fig. 3.9 and equations (3.26) to (3.28) is the core part of multi-head attention described in section 3.8.4. Its job is to use the keys to find the positions most relevant to the query and amplify the values at these positions. It is run in parallel for each of the n_{heads} attention heads. For simplicity, the head index h is not repeated in the diagrams and equations here. Note that scaled dot-product attention does not have any learnable parameters, the learnable parts like the creation of query, key and value vectors and projection after merging heads is done by multi-head attention. The attention mechanism described here is more a less a more detailed explanation of the overview given in section 2.9:

1. The query vector $\mathbf{q}_i$ at position i is compared with the key vector of each position $1 \leq i' \leq i$ using dot product to generate a score value $w_{i'i} \in \mathbb{R}$, as described in eq. (3.26). The scaling factor $\frac{1}{\sqrt{d_{head}}}$ helps improve gradient stability [1, p. 4].

   Positions after i ($i' > i$) are not considered because of the autoregressive nature of the model, i.e. the model is going to be used to predict the token at position $i+1$ based on the tokens up to position i. While the entire sequence is available during training, allowing for an efficient training process, during text generation only the tokens up to the current position i will be available so this is what should be learned [1, p. 5].

2. The scores $w_{i'i}$ are normalized in eq. (3.27) using softmax so their sum is 1, this is required for the next step.

3. Using the scores from before as weights, the output vector $\mathbf{a}_i$ is "blended together" in eq. (3.28) from each of the value vectors $\mathbf{v}_{i'}$ [48]:

$$w_{i'i} = \frac{1}{\sqrt{d_{\text{head}}}} \mathbf{k}_{i'} \mathbf{q}_i \tag{3.26}$$

$$\text{softmax}(w_{i'i}) = \frac{\mathrm{e}^{w_{i'i}}}{\sum_{1 \le i'' \le i} \mathrm{e}^{w_{i''i}}} \tag{3.27}$$

$$\mathbf{a}_i = \sum_{1 \le i' \le i} \text{softmax}(w_{i'i}) \mathbf{v}_{i'} \tag{3.28}$$

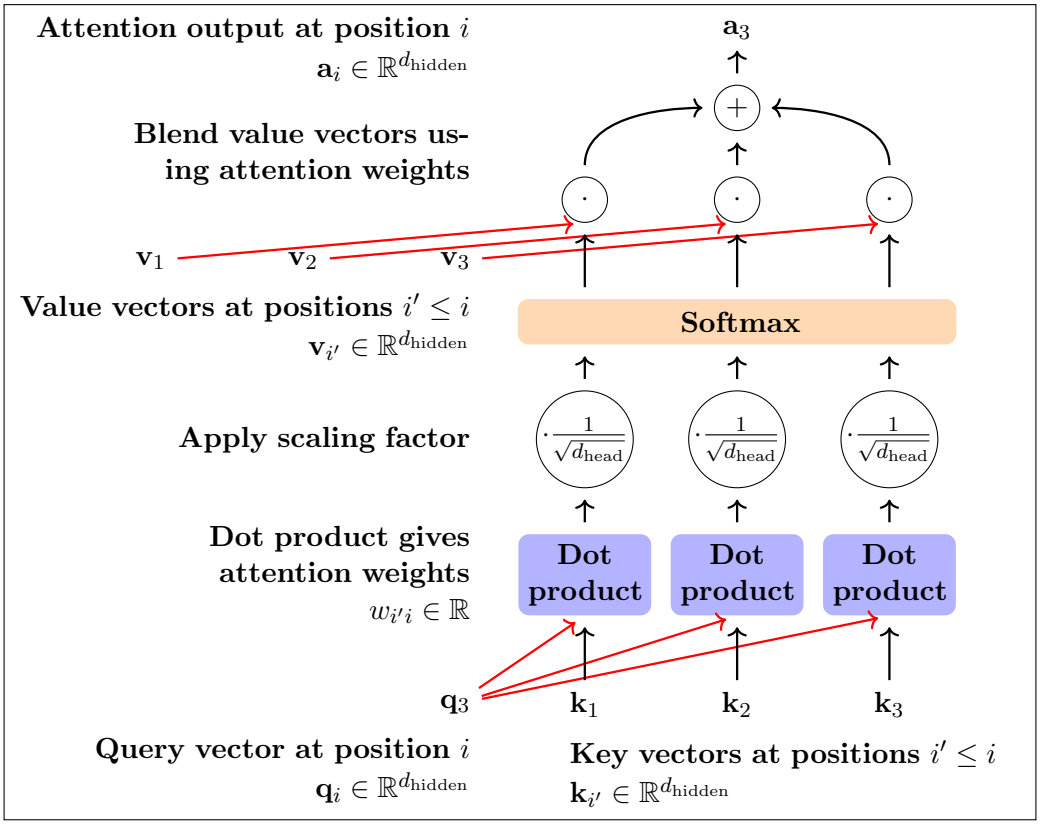

Figure 3.9: Structure of the scaled dot-product attention mechanism. This diagram illustrates the computation of the attention output $\mathbf{a}_3$. The other output positions $\mathbf{a}_i$ are computed in a similar fashion, replacing $\mathbf{q}_3$ in the diagram with $\mathbf{q}_i$ and considering $\mathbf{k}_1 \ldots \mathbf{k}_i$ and $\mathbf{v}_1 \ldots \mathbf{v}_i$, giving the diagram i columns.

3.8.6 Multi-layer perceptron

The multi-layer perceptron stage in the GPT-2 block is a simple feed-forward neural network with one hidden layer [1, p. 5], illustrated in fig. 3.10 and equations (3.29) to (3.31).

In contrast to the original transformer paper, GPT-2 uses the Gaussian Error Linear Unit (GELU) activation function [38, p. 5]. The GELU function for a real input z is defined in eq. (3.29) as z multiplied by the probability that a random variable Z which follows a normal distribution with mean 0 and variance 1 is less than z [41].

The input vector $\mathbf{h}_i$ is first subjected to a linear transformation with weights $\mathbf{W}_{\text{MLP1}}$ and biases $\mathbf{b}_{\text{MLP1}}$, followed by applying GELU to each component, to get the intermediate $\mathbf{h}^{[1]}{}_i$, as described by eq. (3.30). The size of this intermediate vector is dependent on the dimensions of the learnable weight matrices and bias vectors $(\mathbf{W}_{\text{MLP1}}, \mathbf{b}_{\text{MLP1}}), (\mathbf{W}_{\text{MLP2}}, \mathbf{b}_{\text{MLP2}})$, which are typically configured such that the dimension of $\mathbf{h}^{[1]}{}_i$ is $4 \cdot d_{\text{hidden}}$ [1, p. 5] [47, source code]. Then another linear transformation happens in eq. (3.31) with a different set of learnable parameters $\mathbf{W}_{\text{MLP2}}, \mathbf{b}_{\text{MLP2}}$ to get the output $\mathbf{h}'_i$ with dimension d_{hidden}.

$$\text{GELU}(z) = z \cdot \mathbb{P}(Z < z) \qquad z \in \mathbb{R}, Z \sim \mathcal{N}(0,1) \tag{3.29}$$

$$\mathbf{h}^{[1]}{}_i = \text{GELU}(\mathbf{W}_{\text{MLP1}}\mathbf{h}_i + \mathbf{b}_{\text{MLP1}}) \tag{3.30}$$

$$\mathbf{h}'_i = \mathbf{W}_{\text{MLP2}}\mathbf{h}^{[1]}{}_i + \mathbf{b}_{\text{MLP2}} \tag{3.31}$$

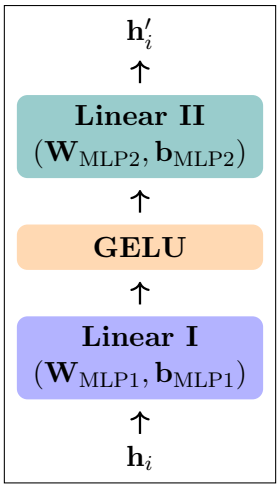

Figure 3.10: Multi-layer perceptron stage of the GPT-2 block. It is a simple feed-forward neural network with two linear steps with bias and an activation function in between.

4 Arithmetic using GPT-2

Building on the results of [7], we wanted to replicate their good results for a small transformer model learning addition with up to 10 decimal digits [7, p. 15]. We also wanted to focus on the harder task of integer multiplication with up to 10 decimal digits, which was studied in [7] only for 2 decimal digit multiplicands. Because we achieved no success in training this operation using our baseline setup (section 5.4), we experimented with a number of approaches that could help and are illustrated in this section: From simpler approaches like changing the distribution of samples in the training dataset (section 4.4), to more elaborate ones like automatically generated scratchpads (description of intermediate steps of a calculation) that augment the training samples (section 4.6).

[7] also studied the (irrational) square root function, with the output being approximated by its first five decimal digits. We wanted to stick to integer operations and thus included the integer square root function (section 4.3.1) to our studied operations to see how hard it is to learn compared to addition and multiplication, and how it would respond to the techniques we tried for improving multiplication performance.

4.1 Tokenization

We stick to a simple tokenization scheme where each character in the string is converted to one token ID based on its ASCII value, meaning our model supports token IDs from 0 to 127, thus $d_{\text{vocab}} = 128$ for all our models. Token ID 0 is reserved to denote the end of the token sequence, it is appended to the ASCII token sequence.

```
"1+2=3" -> [49, 43, 50, 61, 51, 0]
```

4.2 Model choice

We use a transformer model with mostly the same GPT-2 architecture described in chapter 3, with parameters $d_{\text{vocab}} = 128, n_{\text{blocks}} = 6, d_{\text{hidden}} = 384, n_{\text{heads}} = 6, d_{\text{head}} = 64$ (these parameters are explained in section 3.2). This means we use the same model architecture and configuration parameters as in [7] and thus get a model with about 11M parameters [7, p. 3]. This is considerably smaller than the versions of GPT-2 released by OpenAI with about 100M to 1B parameters (section 3.2). Since our resources in terms of hardware are limited and we train for single-purpose transformer models that can only do one specific arithmetic task, we consider this acceptable. We do however briefly experiment with larger model sizes in some experiments described in section 5.5.

The maximum sequence length of the model, n_{pos}, gets adjusted based on the string lengths of the training data for that particular experiment.

4.3 Arithmetic operations

We study the performance of transformer models on three different arithmetic operations: addition, multiplication and integer square root. Addition and multiplication are standard arithmetic operations and common building blocks of more complex operations like exponentiation and evaluation of polynomial functions. Their learnability by transformer models has been investigated in [7] and [6] among others. Integer square root, on the other hand, has to our knowledge not been studied before in the context of transformer networks. This operation is explained in more detail in section 4.3.1. For all operations and experiments, operands are represented

as a sequence of decimal digits. The training sample strings for each arithmetic operation are structured as follows:

- **Addition:** Let a, b be positive integers and let $c = a + b$. The model is then trained on strings of the form $a + b = c$.
- **Multiplication:** Let a, b be positive integers and let $c = a \cdot b$. The model is then trained on sequences of the form $a \cdot b = c$.
- **Integer square root:** Let k be a positive integer. We train the model of sequences of the form $x : k$ where k is the integer square root of x.

4.3.1 Integer square root

In addition to addition and multiplication, we also study the learnability of computing square roots. Instead of the regular square root function with irrational values, we use integer square root. This is consistent with the addition and multiplication operations in that it is an integer to integer mapping, with exactly one correct, finite answer for each input number, yet being equivalent to approximating the regular square root function.

The integer square root function $\lfloor\sqrt{x}\rfloor$ is defined as $\max(\{k \in \mathbb{N} | k^2 \leq x\})$. A model that can compute the integer square root $\lfloor\sqrt{x}\rfloor$ for integers x with a certain number of decimal digits can also approximate the regular square root function, as shown in this example about computing an approximation of $\sqrt{2}$:

$$\begin{aligned}
\sqrt{2} &= 10^{-5}\sqrt{2 \cdot 10^{10}} \\
\lfloor\sqrt{2 \cdot 10^{10}}\rfloor &= 141421 \\
10^{-5}\lfloor\sqrt{2 \cdot 10^{10}}\rfloor &= 1.41421 \\
\sqrt{2} &\approx 1.4142135624
\end{aligned}$$

4.4 Sampling methods

The distribution from which training samples are picked can have a big impact on the performance of machine learning models. To find out how much of an improvement can be had on the arithmetic tasks studied by us just by changing the sampling method, we experimented with multiple sampling methods: Basic sampling, from-zero sampling, uniform digits sampling and uniform bits sampling. With the exception of the sampling strategy experiments (section 5.6) where we compare each of the four sampling methods, we stick to basic sampling for all experiments. The sampling methods are defined as follows:

- **Basic sampling:** For the addition and multiplication tasks with n digits, the operands a and b are independently sampled uniformly from the range $[10^{n-1}, 10^n - 1]$, which ensures both operands have exactly n decimal digits. We train the models for $n = 3$, $n = 5$, and $n = 10$ operand digits, giving a sample space of approximate size 10^6, 10^{10}, 10^{20} respectively.

  For the integer square root operation with n operand digits, we also sample the operand x uniformly from the range $[10^{n-1}, 10^n - 1]$. However, because square root is a unary operation, we double the number of decimal digits of the single operand to make sure we have a similarly large space of possible samples. Therefore we train the models for $n = 6$, $n = 10$, and $n = 20$ operand digits, giving a sample space of approximate size 10^6, 10^{10}, 10^{20} respectively, similar to the sample space sizes for addition and multiplication.

- **From-zero sampling:** For basic sampling, the limits of operand sampling were set as $[10^{n-1}, 10^n - 1]$ to include all n-digit numbers, where n is the number of operand digits. From-zero sampling instead uses the limits $[0, 10^n - 1]$, thus also including operands with fewer than n decimal digits.

- **Uniform digits sampling:** For this sampling method, first a number of decimal digits k is picked uniformly at random from $[1, n]$. Then the operands are picked uniformly at random from $[10^{k-1}, 10^k - 1]$, to ensure they are k-decimal digit integers.

- **Uniform bits sampling:** Let p be the smallest integer so that $2^p \geq 10^n$. This ensures all numbers with n decimal digits are covered by the range $[1, 2^p]$. A number of bits k is picked uniformly at random from $[1, p]$. Then the operands are picked uniformly at random from $[2^{k-1}, 2^k - 1]$ to ensure they are k-bit integers.

4.5 Result reversal

Providing some intermediate steps how to arrive at the result within each training sample as opposed to just input/output pairs can make a substantial improvement to final accuracy when transformer models learn addition, especially with more operand digits ([7, p. 15]). Similar to [7], we tested two different strategies: Reversing the digits of the result, and detailing a step-by step arithmetic process to compute the result, similar to doing addition or multiplication on paper or in an algorithm. This we refer to as a "scratchpad". We expanded the experiments in [7] beyond addition to multiplication and integer square root with varying operand sizes, testing whether result reversal or scratchpad could also help with learnability for these harder operations.

When computing a sum or product by hand, one starts from the right, computing the least significant digits of the result before moving left to the more significant digits. However, in the baseline experiments, the model needs to predict the most significant digit first and the least significant last.

To accurately predict the most significant digit of a sum, one generally needs to have the exact result of the sum computed already (think about operands where the sum is close to wrapping on the most-significant digit, i.e., small changes in the operand can cause a change in the most significant digit of the sum).

In contrast, knowing the least significant digits of the operands is sufficient to compute the least significant digit of the sum. Thus, it seems reasonable to assume that computing a sum "backwards" would be easier for a model than computing it in normal digit order.

To allow the model to first write down the reversed sum before "committing" to the actual result, we modify the training samples: After the = sign, which is the point from which the model is asked to predict the rest of the training sample (which is the sum that we want to know to test addition capability), we put the reversed digits of the sum in brackets before the normal sum. For example, the training sample `456+789=1245` would be transformed into `456+789=[5421]1245`:

For the multiplication and square root tasks, the digits of the product or integer square root are reversed in the same fashion - first writing the reversed result in brackets before writing the actual result.

4.6 Scratchpads

Compared to augmenting the training sample with the reversed result, which is a simple process that does not tell the model in detail how to arrive at the result, a scratchpad is a detailed, step-by-step, digit-by-digit walkthrough of a basic algorithm solving the task, decomposing it into small intermediate chunks.

Similar to result reversal, we put the scratchpad (the intermediate steps) in brackets before the correct result. The model then, when prompted, outputs this scratchpad for the task at hand, thus executing the algorithm, then in the ideal case arrives at the correct output after the brackets.

4.6.1 Addition

For the addition task, the scratchpad illustrates basic digit-by digit addition of two integers: Add two digits of operands plus carry from the previous step, separate ones and tens, take ones as a digit of the sum, keep tens as carry. Move one digit to the left and repeat until the operands have been fully processed. For the training sample `456+789=1245` the sample with scratchpad is as described in listing 4.1.

Each step is separated by pipe(`|`) characters. The two summand digits, previous carry, sum digit, and new carry are written for each step. The scratchpad is deliberately kept minimal because of the increase in training time with increased sample length.

This increase in training time is not due to an increase in model parameters, but rather due to the model's attention mechanism comparing one position to all previous positions (section 3.8.4). This causes training time to increase quadratically with sequence length. Memory requirements also increase quadratically due to the resulting attention matrix when comparing each position with each previous one.

```
456+789=[69051|58141|47121]1245
```

Listing 4.1: Example scratchpad for the addition task 456+789=1245.

4.6.2 Multiplication

Basic multiplication works by doing single-digit multiplication between the multiplicand and all digits of the multiplier, then adding an appropriate amount of zero digits to each intermediate product, then computing the sum of all the intermediate products. This process forms the basis for our multiplication scratchpad. For example, the multiplication task `67*89=5963` is transformed into the scratchpad-augmented form seen in listing 4.2. The multiplication scratchpad is divided by `|` into the part computing the digit products and the part doing the final addition. Each digit product has its own scratchpad enclosed in `{}`.

```
67*89=[
  *80{78065,68535}5360,
  *9{79036,69606}603
  |
  {03,0,3,0;60,0,6,0;36,0,9,0;5,0,5,0}
]5963
```

Listing 4.2: Example scratchpad for the multiplication task 67*89=5963.

The computation of the digit products happens like this (example for the multiplication 67*89):

```
*80{78065,68535}5360
```

First comes the multiplier (80 in this case), then the multiplication steps in curly braces, then the result of the digit multiplication. For each multiplication step, we list five digits: First, the corresponding digit of the multiplicand (first 7, then 6 for 67), then the digit of the multiplier (always 8 for 80), then the previous carry, then the output digit, then the current carry.

The final addition happens like this ($5360 + 603 = 5963$ for our example):

```
{03,0,3,0;60,0,6,0;36,0,9,0;5,0,5,0}
```

Each addition step is separated by semicolons. The parts of each addition step are separated by commas. For each addition step, we first list the corresponding digits of the summands (e.g., 0 and 3 for the first step of $5360 + 603$). Then we list the previous carry, then the output digit, and finally, the current carry.

4.6.3 Square root

A straightforward way to compute the integer square root of an integer n is by using a binary search algorithm is illustrated in listing 4.3.

```
low, high = 0, n
while high - low > 1:
  mid = floor((low + high) / 2)
  if mid*mid <= n:
    low, high = mid, high
  else:
    low, high = low, mid
if high * high <= n:
  return high
else:
  return low
```

Listing 4.3: Binary search algorithm for computing the integer square root.

The integer square root scratchpad we use follows the steps of this algorithm, illustrated on the training sample `123:11` in listing 4.4.

```
123:[
  {0,123} 61*61=3721|
  {0,61} 30*30=900|
  {0,30} 15*15=225|
  {0,15} 7*7=49|
  {7,15} 11*11=121|
  {11,15} 13*13=169|
  {11,13} 12*12=144|
  {11,12}
]11
```

Listing 4.4: Scratchpad example for computing the integer square root of 123.

The steps of the binary search algorithm are in brackets, separated by pipes. Consider the first step for example:

`{0,123} 61*61=3721`

The current search range is in braces, for this step it is from 0 to 123. Then, the midpoint of the search range (61) is squared to 3721. We are looking for the integer square root of 123, which is the largest integer whose square is at most 123 (section 4.3.1). Because 3721 is larger than $n = 123$, the search range in the next step gets restricted to the lower half, which is from 0 to 61. The algorithm ends when the search range consists of at most two integers (11 and 12 in this case), at which point the larger one is tested, as $12 * 12 = 144$ is too large, 11 is the correct integer square root.

It would have been nice to include the multiplication scratchpad from section 4.6.2 for the square operations of this algorithm, however to keep training feasible we have to keep the samples below a few thousand characters. Considering the fact that we want to compute square roots of 6, 10, and 20 digit integers as in the baseline experiments, this is not possible. Thus we simply include the square operations without their own scratchpads and hope this illustration of binary search still provides improved accuracy compared to the baseline.

4.7 Attention heatmaps

To get some degree of insight into how the trained model computes the results for the arithmetic tasks presented to it, we generated 2D heatmaps showing activation of the attention mechanism for various input sequences. Our process for creating heatmap diagrams roughly follows the attention heatmaps presented in [29] and [30]:

The model consists of a pre-processing stage, a number of GPT-2 blocks and finally a post-processing stage (fig. 3.1). Inside the GPT-2 blocks, the core part is the attention mechanism (section 3.8.5): The input vector at each position is transformed into a query, a key and a value vector. The query vector at one position is then compared with the key vectors at every position, to generate an attention score from 0 to 1 at each position, which is then used to blend the value vectors together to get the output at this position (fig. 3.9). Before running the attention mechanism, the input vector is split into multiple attention heads, on each of which a separate attention mechanism is run, whose outputs are then combined (fig. 3.8).

We can thus generate, for a given token sequence, a diagram that, for each GPT-2 block and each attention head inside that block, contains a 2D heatmap that shows which token / position attends to which other position. We create two types of heatmap diagrams: First we started with full diagrams showing each block and attention head, then for clarity we simplified the other diagrams to averaged diagrams with just a single 2D image:

- **Full diagram:** For each GPT-2 block of the model and each attention head per block, generate a 2D image with color values corresponding to how strongly attention output for one position is influenced by inputs at each position (section 3.8.5).

- **Averaged diagram:** When looking at the full diagrams, we noticed meaningful attention patterns only in the first GPT-2 block, with no clear difference between attention heads. Also, full diagrams can get quite large, especially when showing the model's handling of longer sequences (result reversal or scratchpad). We thus reduced diagrams to only showing the first layer and averaging among all attention heads. This is in contrast to [29] which focused more on the different roles of individual attention heads. For out purposes of showing some meaningful patters in the model's attention mechanism, averaging worked fine however and presented a single 2D image.

Examples of our full and averaged heatmap diagrams can be seen in section 6.6.

5 Experimental setup

Based on the methods outlined in chapter 4, we set up a training pipeline that first creates a dataset for the given arithmetic operation, operand digits and sampling method, then initializes a GPT-2-style model (section 4.2) with the given model configuration, and then trains the model on the dataset, monitoring loss and accuracy during training. The details of our training pipeline are as follows:

- **Libraries/Frameworks:** We use the HuggingFace transformers library [53] with PyTorch backend [31] which takes care of implementing transformer networks and the training process. Based on this, we wrote some Python code that implements generating training samples, running model training, evaluating model performance, logging results and generating plots. The generation of plots from the result logs utilized the matplotlib Python library.

- **Hardware:** The experiments were run on Linux machines with NVIDIA GPUs that were rented and accessed via a GPU renting website. We ensured each system had a reasonably recent NVIDIA GPU with at least 16GB VRAM to ensure our experiment computations fit inside GPU memory.

- **Model implementation:** We use the GPT2LMHeadModel class of the HuggingFace transformers library which implements a GPT-2-style decoder-only transformer model. We adjust the model configuration so it matches nanoGPT [52], a basic lightweight GPT-2 implementation which was also used for the experiments in [7]. The model uses $d_{\text{hidden}} = 384$, $n_{\text{blocks}} = 6$, $n_{\text{heads}} = 6$.

- **Weight initialization:** The model weights are initialized using the default distributions as defined in the GPT2LMHeadModel class of the HuggingFace transformers framework.

- **Optimizer:** Training uses the AdamW optimizer with default parameters as defined by the HuggingFace transformers Trainer / TrainingArguments class.

- **Batch size:** For the baseline experiments, we used a fixed batch size of 256. We picked this batch size as it was also used in [7] and it fits the memory of our training GPU. For some experiments (section 5.8) we needed to reduce batch size to fit batches into GPU memory.

- **Learning rate scheduling:** The learning rate is adjusted using a linear schedule with warmup steps, with default parameters as defined by the HuggingFace transformers framework. For computing the learning rate, we assume a maximum of 200 epochs for training.

- **Early stopping:** We use an early stopping strategy where training is aborted if for 5 epochs there is no new minimum in loss on the validation dataset (minimum improvement $\epsilon = 10^{-4}$).

- **Text generation:** To generate answers to a prompt string like "1+2=", we use autoregressive generation as is standard practice with transformer decoder models (section 2.6). We use greedy decoding, always selecting the token with the highest probability. Generation stops when token ID 0 (stop token) is generated or the model maximum sequence length is reached.

5.1 Datasets

Based on the sampling and formatting methods described above, for each experiment we create three separate dataset:

- **Training dataset:** This dataset is used to optimize the model weights during training. Its size varies from 1k to 100k samples as part of the experiment.
- **Validation dataset:** A separate dataset with 1k samples. Loss on this dataset is used to decide when to stop training (early stopping). Accuracy on this dataset is also tracked. For some experiments (section 5.8) we had to lower the size of the validation dataset to avoid accuracy tracking slowing down training too much.
- **Test dataset:** Another separate dataset with 1k samples. This dataset is used at the end of each training run to compute the final accuracy numbers.

All datasets are sampled using the **basic sampling** method illustrated in section 4.4. This applies to all experiments except those studying different sampling methods (section 5.6). For addition and multiplication, the operands a and b of each sample are encoded into a string based on their decimal digit representation, the same applies to the sum / product c. To avoid uneven string/sequence length, c is encoded with leading zeroes up to the maximum possible sum / product length based on the operand size. For example, for 3-digit addition, the sum is padded to 4 digits:

```
"123+456=0579"
```

For the square root operation, we encode a sample like this:

```
"500000:707"
```

For even n, the integer square root of a n-digit decimal number always has $\frac{n}{2}$ decimal digits, thus no padding is needed.

5.2 Evaluation

During the training process, multiple metrics are tracked:

- **Average batch loss:** For each training sample, the model outputs a token probability distribution at each position of the sample. This is compared with the correct next token at this position to compute a cross-entropy loss number. This loss is averaged among all positions of a sample and all samples in the current batch. To reduce statistical noise, the HuggingFace trainer class also averages these batch loss numbers among multiple consecutive training steps.
- **Validation loss:** At the end of each epoch, the average loss over all samples in the validation dataset is computed. This number is used to track training progress and stop training if no more progress is made.
- **Validation accuracy:** For each sample in the validation dataset, the string is split at the "=" character, yielding a prompt and answer string (e.g. prompt "1+2=" and corresponding answer "3"). For the prompt string, the model's answer is generated as described in section 3.6. This answer is then compared to the correct answer. The ratio of samples where the model's answer matches the correct answer gives the accuracy.
- **Test accuracy:** It is evaluated just like validation dataset accuracy, but on the test dataset. This gives the final accuracy numbers reported in the tables below.

5.3 Error analysis

We wanted to analyze not just the performance of the final models in terms of the percentage of correctly solved tasks, but also take a look at the types of errors the model makes. For this, we picked the first few tasks from the test dataset that the model answered incorrectly. If there were no errors in the test dataset, we created an additional error analysis dataset with 100k samples to look for rare errors.

If there were errors, we then created a table with a few of the error samples for the experiment, with columns for the input (e.g., "123+456"), expected output (e.g., "0579"), model output (e.g., "0679") and additional remarks for some experiments. For longer output sequences (e.g., scratchpad experiments) the erroneous part of the model output was highlighted. We then tried to identify and discuss observable patterns in the small sample of errors.

5.4 Baseline experiments

To test the ability of transformer networks to learn arithmetic operations, and to see whether we can replicate the results of [7], we first run some baseline experiments. The training pipeline is as described above and the training samples for the studied operations were created as described in section 4.3.

We run a total of 3*3*3=27 baseline experiments, comparing 3 operations, 3 operand sizes per operation, and 3 training dataset sizes:

- **Addition:** 3, 5, or 10 operand digits, 1k, 10k, or 100k training dataset.
- **Multiplication:** 3, 5, or 10 operand digits, 1k, 10k, or 100k training dataset.
- **Integer square root:** 6, 10, or 20 operand digits, 1k, 10k, or 100k training dataset.

Because integer square root is a unary operation, we double the number of operand digits compared to addition or multiplication.

5.5 Model size experiments

To see whether changing the number of parameters of the model improves accuracy, we tried each of the following changes to model dimensions:

- Change the hidden vector size d_{hidden} from 384 to either 192 or 768. This causes a substantial change to the number of learnable model parameters: The model with $d_{\text{hidden}} = 192$ has about 2.7 million parameters, the baseline model with $d_{\text{hidden}} = 384$ about 11M parameters, and the model with $d_{\text{hidden}} = 768$ about 43 million parameters. Thus doubling d_{hidden} roughly quadruples the number of parameters (due to matrices for transformations on vectors twice the size having 4 times as many parameters).
- Change the number of GPT-2 blocks n_{blocks} from 6 to either 3 or 12. This also changes the number of parameters, but in a linear fashion: The model with 3 blocks has 5.4 million parameters, and the model with 12 blocks has 21 million parameters.
- Change the number of attention heads n_{heads} from 6 to either 3 or 12. This has no effect on the number of model parameters, it only affects the "partitioning" of the parameters inside the model.

We did not try to combine these changes, each experiment only had one parameter of the model architecture changed. Due to time/hardware constraints, we limited experiments to the following tasks:

- **Addition:** 10 operand digits, 100k training dataset.
- **Multiplication:** 5 operand digits, 100k training dataset.

We picked these two tasks from the baseline experiments because addition with 10 digits could be learned well with the 100k dataset, while still being harder than 3- or 5-digit addition, allowing more chances to see a difference in model performance in terms of how many optimization steps it takes to converge. Similarly, multiplication with 5 digits and 100k training dataset was chosen because the baseline model could not learn it and achieved zero accuracy. At the same time, learning 5-digit multiplication seemed somewhat realistic with a more capable model architecture, considering that 3-digit multiplication could be learned at a low but not zero (22%) accuracy.

In summary, we run these 12 experiments with 6 different model changes attempted on two operations and fixed operand digits and dataset size per operation:

- **Addition:** 10 operand digits, 100k training dataset, baseline model but with one of these changes: $d_{\text{hidden}} = 192, d_{\text{hidden}} = 768, n_{\text{blocks}} = 3, n_{\text{blocks}} = 12, n_{\text{heads}} = 3$ or $n_{\text{heads}} = 12$.
- **Multiplication:** 5 operand digits, 100k training dataset, baseline model but with one of these changes: $d_{\text{hidden}} = 192, d_{\text{hidden}} = 768, n_{\text{blocks}} = 3, n_{\text{blocks}} = 12, n_{\text{heads}} = 3$ or $n_{\text{heads}} = 12$.

5.6 Sampling strategy experiments

Changing the sampling method / distribution of training samples does not change the length of the training sequences. We can thus run the same 27 baseline experiments (section 5.4) with each of the four sampling methods outlined in section 5.6 except "Basic sampling" which was used for the baseline experiments. Thus we run 27*3=81 additional experiments to evaluate sampling strategies:

- **Addition:** 3, 5, or 10 operand digits, 1k, 10k, or 100k training dataset, from-zero, uniform-digits or uniform-bits sampling.
- **Multiplication:** 3, 5, or 10 operand digits, 1k, 10k, or 100k training dataset, from-zero, uniform-digits or uniform-bits sampling.
- **Integer square root:** 6, 10, or 20 operand digits, 1k, 10k, or 100k training dataset, from-zero, uniform-digits or uniform-bits sampling.

5.7 Result reversal experiments

We wanted to find out whether reversing the digits of the result of the arithmetic tasks in the training samples can improve model performance. For this we augmented the training samples with the reversed digits of the result, as described in section 4.5.

This results in increased sample length (table 5.1) which means we have to increase the number of positions in the model configuration (n_{pos}) to handle these longer sequences. This does lead to more model parameters due to each position having a learnable positional embedding vector (section 3.4). The increase however is just a minute fraction of the total parameters (e.g., 10.710 million parameters for the 10-digit addition baseline requiring 34 positions, and 10.715 million parameters for 10-digit addition with result reversal requiring 47 positions). This minute increase does not necessitate other changes to the training regime such as reduced batch sizes or fewer epochs - we can keep the default batch size of 256 and epoch limit of 200 (chapter 5).

For the result reversal experiments, to evaluate accuracy as described in section 5.2 we do not consider the entire generated output of the model (the reversed result in brackets and the final result), but instead strip the reversed part before only comparing the actual result.

Task	Sequence length (baseline)	Sequence length (result reversal)
Addition (3 digits)	13	19
Addition (5 digits)	19	27
Addition (10 digits)	34	47
Multiplication (3 digits)	15	23
Multiplication (5 digits)	23	35
Multiplication (10 digits)	43	65
Square root (6 digits)	11	16
Square root (10 digits)	17	24
Square root (20 digits)	32	44

Table 5.1: Sequence lengths for training samples (prompt string + response string + end token) for various tasks and operand sizes.

We can thus repeat the panel of 27 baseline experiments at the same settings (section 5.4), with the training samples augmented with result reversal:

- **Addition:** 3, 5, or 10 operand digits, 1k, 10k, or 100k training dataset, using result reversal.
- **Multiplication:** 3, 5, or 10 operand digits, 1k, 10k, or 100k training dataset, using result reversal.
- **Integer square root:** 6, 10, or 20 operand digits, 1k, 10k, or 100k training dataset, using result reversal.

5.8 Scratchpad experiments

Since scratchpads (including intermediate steps of the computation in the training sample) did show promise in [7] and [34], we also wanted to test this for our panel of operations and operand sizes.

Augmenting the training samples with a scratchpad that details the steps of the arithmetic operation to be performed results in a large increase in sequence length (table 5.2). This increase is especially noticeable for the multiplication operation (which is composed of several single-digit multiplications followed by a large addition) and the square root operation (which is done by binary search with squaring at every step).

Task	Sequence length (baseline)	Sequence length (scratchpad)
Addition (3 digits)	13	32
Addition (5 digits)	19	50
Addition (10 digits)	34	95
Multiplication (3 digits)	15	159
Multiplication (5 digits)	23	356
Multiplication (10 digits)	43	1181
Square root (6 digits)	11	602
Square root (10 digits)	17	1447
Square root (20 digits)	32	5083

Table 5.2: Sequence lengths for training samples (prompt string + response string + end token) for various tasks and operand sizes.

These large sequences of sometimes well beyond 1000 characters necessitate some changes to the training regime. Based on the available hardware, we need to make training batches and the intermediate results during forward/backward pass fit into GPU memory. The smaller batches in turn mean more optimization steps per epoch, thus taking more time to do training for a given number of epochs. At the same time, a training batch with longer sequences results in larger matrices during optimization, thus taking more time per batch than the same batch size with shorter sequences. Time for evaluating the model (generating answers for the samples in the test/validation dataset to test accuracy) also rises proportionally to the lengths of the sequences being generated, meaning it can become a significant part of training time when sequences are long due to an elaborate scratchpad being trained on.

We want to keep the duration for a single experiment (e.g., square root, using scratchpad, 20 digits, 100k dataset) below 24 hours on the hardware available to us. We still want to run all experiments, even those with the largest dataset size of 100k, thus we made the compromise of reducing the maximum number of training epochs for the experiments that would take too much time otherwise. This does have the downside of the models being undertrained (unless early stopping would have halted training before the epoch limit) but it seems like the best choice given the time and hardware constraints.

For the addition operation (table 5.3) the only change that needed to be made was reducing the maximum epochs from 200 to 20 for experiments with the large 100k training dataset.

Operand digits, dataset size	**Batch size (baseline)**	**Batch size (scratchpad)**	**Max. epochs (baseline)**	**Max. epochs (scratchpad)**
3 digits, 1k	256	256	200	200
3 digits, 10k	256	256	200	200
3 digits, 100k	256	256	200	20
5 digits, 1k	256	256	200	200
5 digits, 10k	256	256	200	200
5 digits, 100k	256	256	200	20
10 digits, 1k	256	256	200	200
10 digits, 10k	256	256	200	200
10 digits, 100k	256	256	200	20

Table 5.3: Training regime changes for the **addition** operation.

For the multiplication operation (table 5.4) the batch sizes needed to be reduced depending on the sequence length. Also the number of epochs needed to be reduced for the experiments with 10k and 100k datasets.

Operand digits, dataset size	Batch size (baseline)	Batch size (scratchpad)	Max. epochs (baseline)	Max. epochs (scratchpad)
3 digits, 1k	256	128	200	200
3 digits, 10k	256	128	200	50
3 digits, 100k	256	128	200	5
5 digits, 1k	256	64	200	200
5 digits, 10k	256	64	200	50
5 digits, 100k	256	64	200	5
10 digits, 1k	256	16	200	200
10 digits, 10k	256	16	200	50
10 digits, 100k	256	16	200	5

Table 5.4: Training regime changes for the **multiplication** operation.

For the square root operation (table 5.5), due to time and memory constraints (see section 4.6.1), the batch sizes and epoch counts needed to be cut even more, even down to a batch size of 1 and epoch count of 1 for the task with 20 operand digits and 100k dataset. Even though this results in much less training than the comparable baseline task, we still think it is better trying to at least get a glimpse of training results for this task instead of not including it at all.

Operand digits, dataset size	Batch size (baseline)	Batch size (scratchpad)	Max. epochs (baseline)	Max. epochs (scratchpad)
6 digits, 1k	256	32	200	200
6 digits, 10k	256	32	200	50
6 digits, 100k	256	32	200	5
10 digits, 1k	256	8	200	200
10 digits, 10k	256	8	200	50
10 digits, 100k	256	8	200	5
20 digits, 1k	256	1	200	100
20 digits, 10k	256	1	200	10
20 digits, 100k	256	1	200	1

Table 5.5: Training regime changes for the **square root** operation.

In addition to the experiment-specific changes mentioned above, we also reduced the size of the validation dataset from 1000 to 10 elements and perform accuracy evaluations not once every 100 steps during logging but only at the end of each epoch. This gives a less detailed report of accuracy development during training but saves significant evaluation time. Accuracy on the test dataset, which is unchanged at 1000 elements, is performed only at the end of the experiment, same as for the baseline, thus final accuracy numbers in the results table do not suffer in precision.

Similar to the result reversal experiments, we do not consider the entire generated output of the model (the scratchpad and the final result) for evaluating accuracy, but instead strip the scratchpad before only comparing the result.

We thus run the following experiments to test model performance when using scratchpads:

- **Addition:** 3, 5, or 10 operand digits, 1k, 10k, or 100k training dataset, using scratchpad, with modifications to training regime described in table 5.3.
- **Multiplication:** 3, 5, or 10 operand digits, 1k, 10k, or 100k training dataset, using scratchpad, with modifications to training regime described in table 5.4.

- **Integer square root:** 6, 10, or 20 operand digits, 1k, 10k, or 100k training dataset, using scratchpad, with modifications to training regime described in table 5.5.

5.9 Attention heatmaps

We generate diagrams illustrating attention activation between positions (attention heatmap diagrams) based on the methods outlined in section 4.7. All diagrams consider only the first GPT-2 block and are averaged across all attention heads, except for the baseline "123+456" example, where a detailed diagram across all blocks and attention heads is also provided. Diagrams were generated for the following models handling the following example inputs:

- **Addition:**
  - Baseline model trained on 3 digits, 100k training dataset on these examples: "123+456" (both detailed heatmap and averaged heatmap) and "999+001"
  - Model trained using result reversal on 3 digits, 100k training dataset on this example (this includes the model generating the result reversal part): "123+456"
  - Model trained using scratchpad on 3 digits, 100k training dataset on this example (this includes the model generating the scratchpad part): "123+456"
  - Baseline model trained on 5 digits, 100k training dataset on this example: "12345+67890"
- **Multiplication:** Baseline model trained on 3 digits, 100k training dataset on this example: "123*456"
- **Integer square root:** Baseline model trained on 6 digits, 100k training dataset on this example: "123456:"

6 Experimental results

In this section we present the result of running each of the experiments outlined in section 5.4 to section 5.9. Results are presented in the form of colored accuracy tables outlining final model accuracy on the test dataset per experiment. For some experiments, we also plot curves illustrating the development of accuracy during model training. We also sample errors of the trained models for some experiments and compare them to expected results side by side. Finally, this section contains diagrams illustrating attention patterns in the trained models in section 6.6.

6.1 Baseline experiments

For the addition task we can see in table 6.1 that addition up to 10 digits can be learned, as long as the training dataset has at least 10k samples. Accuracy numbers are high, but short of 100% for 5- and 10-digit addition, meaning there are quite a few errors left in the validation dataset. Only 3-digit addition achieved perfect accuracy.

	1k	**10k**	**100k**
3 digits	0.008	1.000	1.000
5 digits	0.000	0.960	0.997
10 digits	0.000	0.826	0.990

Table 6.1: Final model accuracy on the test dataset for the addition operation for the given training dataset sizes and operand digits.

Considering the development of accuracy during training, we can see in fig. 6.1 that accuracy flatlines until a few hundred to a few thousand optimization steps are performed. More operand digits mean more optimization steps to reach the point of accuracy leaving zero. Interestingly, larger dataset sizes also require slightly more optimization steps to leave zero accuracy. After leaving zero, accuracy quickly jumps to about 80%, then slowly keeps climbing to close to 100%. Based on these accuracy curves, it does not seem unreasonable that training for longer that early stopping permits (chapter 5) would have resulted in even better accuracy. Investigating the type and parameters of our early stopping criterion could be worthwhile.

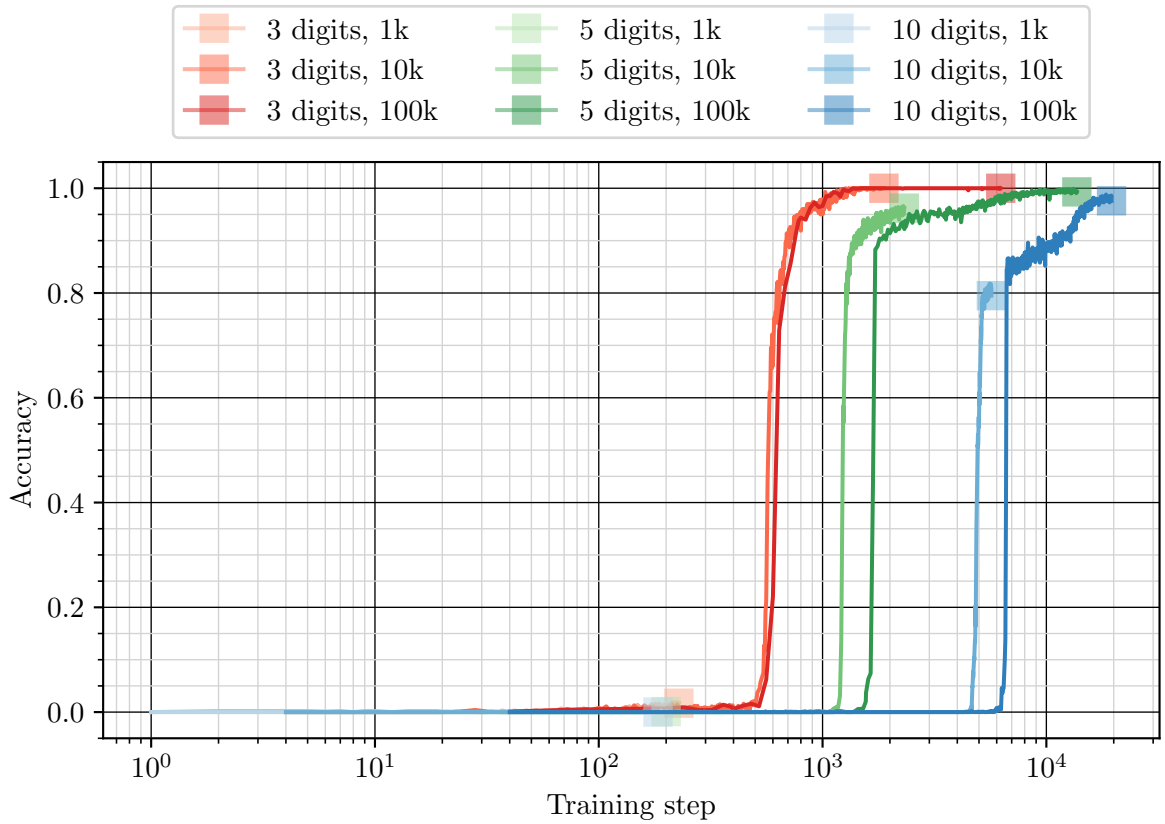

Figure 6.1: Development of accuracy on the validation dataset when training for the addition task for different operand digits and dataset sizes. Square markers represent the point training was terminated by early stopping.

For the multiplication task, we can see in table 6.2 a very different picture to the addition task. Accuracy never gets above zero, except for the 3 digits, 10k dataset experiment, where it reaches about 1%, and the 3 digits, 100k dataset experiment, where it reaches about 22%. This suggests that learning multiplication, all else being equal, is harder than learning addition.

	1k	**10k**	**100k**
3 digits	0.000	0.013	0.218
5 digits	0.000	0.000	0.000
10 digits	0.000	0.000	0.000

Table 6.2: Final model accuracy on the test dataset for the multiplication operation for the given training dataset sizes and operand digits.

When looking at the accuracy curves in fig. 6.2, we can see that for the only experiment where accuracy clearly leaves zero (3-digit multiplication, 100k dataset) accuracy does not jump quickly to a certain value as it did with addition, but slowly keeps climbing until early stopping terminates training. For the experiments that do not achieve any accuracy, we can see that the number of optimization steps performed until training is aborted depends mostly on training dataset size and not much on operand digits, with the experiments with 1k dataset getting aborted after about 200 steps, and the experiments with 100k dataset getting aborted after about 40000 steps.

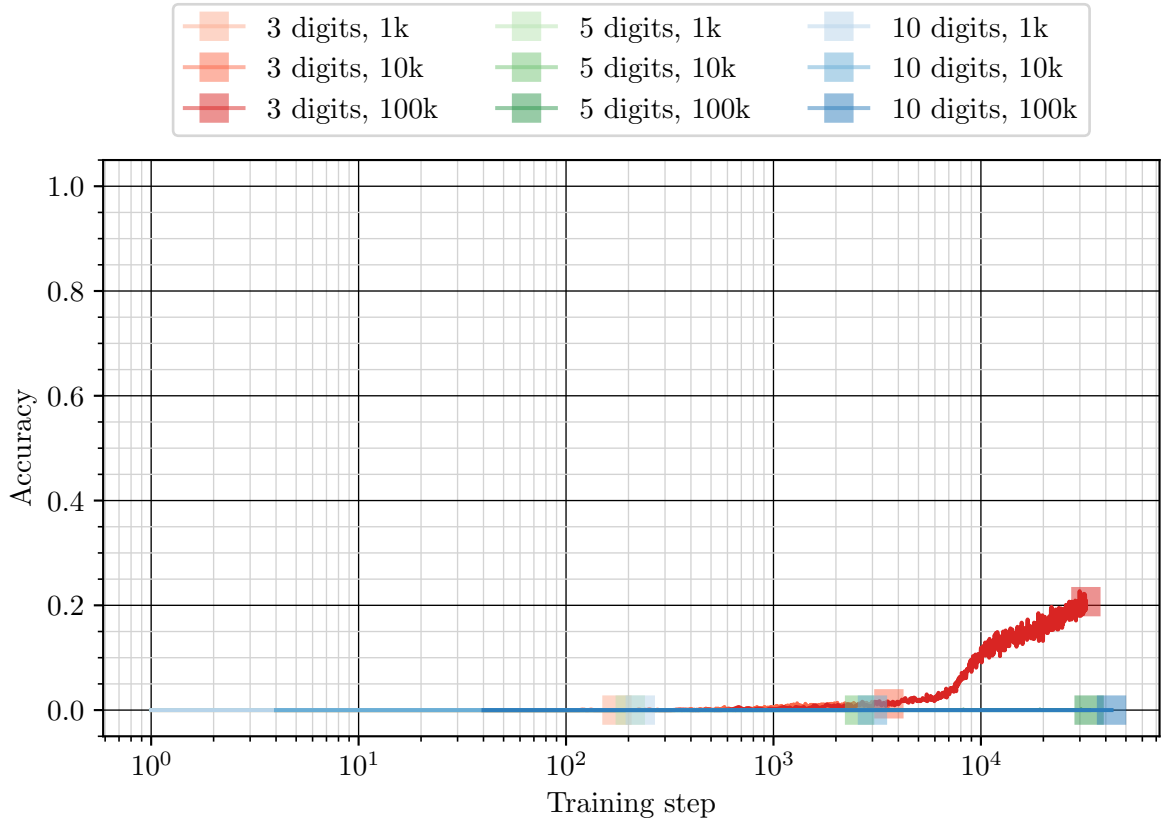

Figure 6.2: Development of accuracy on the validation dataset when training for the multiplication task for different operand digits and dataset sizes. Square markers represent the point training was terminated by early stopping.

For the square root task we can see in table 6.3 that accuracy numbers that lie somewhere between those for addition and multiplication. For 6-digit integer square root, there was high but not perfect accuracy when using a 10k or 100k training dataset. For 10-digit integer square root, accuracy was 63% for the 100k dataset but remained low otherwise. For 20-digit integer square root, no samples in the validation dataset where predicted correctly.

	1k	**10k**	**100k**
6 digits	0.129	0.901	0.983
10 digits	0.004	0.051	0.638
20 digits	0.000	0.000	0.000

Table 6.3: Final model accuracy on the test dataset for the square root operation for the given training dataset sizes and operand digits.

When looking at the accuracy curves in fig. 6.3, we can see curves that are more similar to those of the multiplication experiment than the addition experiments, with accuracy slowly climbing from zero. Once again, it looks like early stopping might prevent the models from achieving their full accuracy potential.

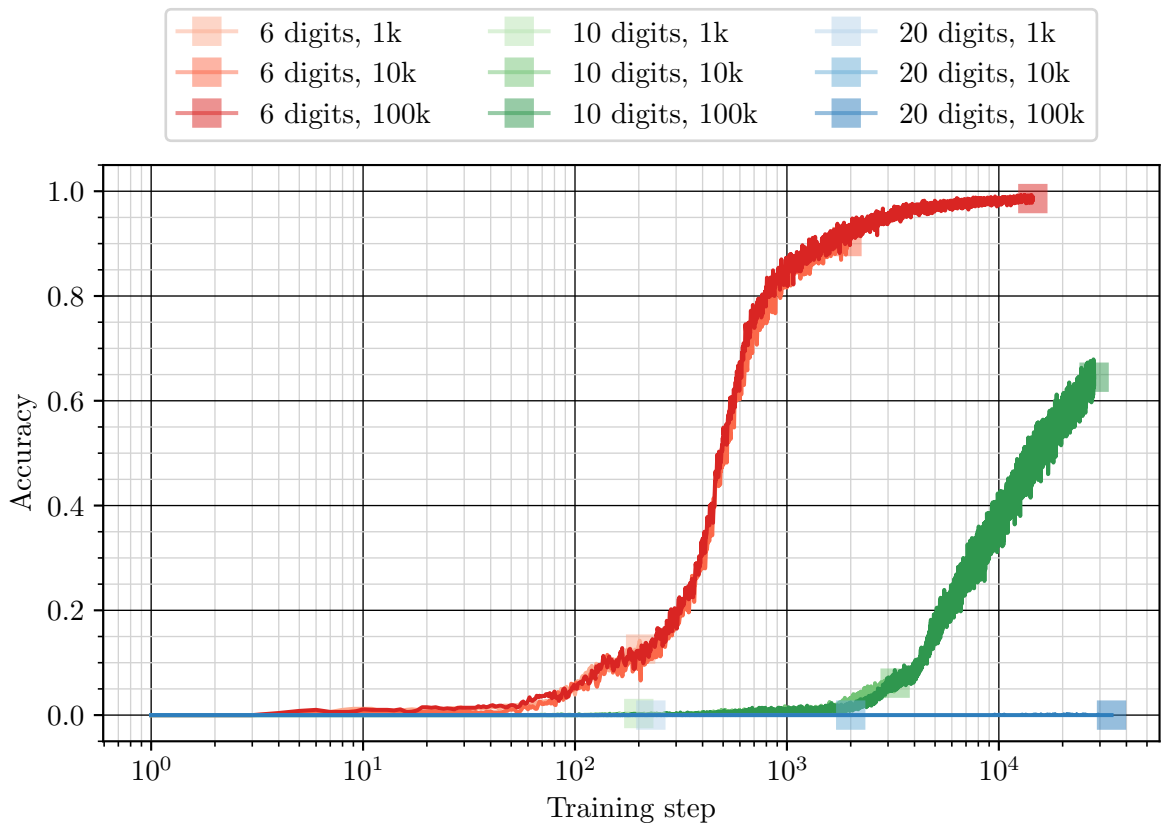

Figure 6.3: Development of accuracy on the validation dataset when training for the square root task for different operand digits and dataset sizes. Square markers represent the point training was terminated by early stopping.

6.1.1 Error analysis

To get a better understanding of the presented accuracy numbers, we took a closer look at samples where the answer of the model differed from the correct one. This was done for tasks where the final model showed perfect or near-perfect accuracy:

- Addition (5 digits, 100k training samples)
- Addition (10 digits, 100k training samples)
- Square root (6 digits, 100k training samples)

We also looked at tasks where the final model showed poor performance or did not even manage to go above zero accuracy:

- Multiplication (3 digits, 100k training samples)
- Multiplication (5 digits, 100k training samples)
- Square root (20 digits, 100k training samples)

For each of these tasks, a separate error analysis dataset of 100k samples was created and the responses of the best model that was saved during training evaluated on it. A selection of the respective errors per task is analyzed below.

For 5-digit addition, the model achieved near-perfect accuracy (table 6.1). It can be seen that the errors the model makes for this task (table 6.4) are single digits of the result being off by one.

Task	Model result	Correct result
91167+53933	145000	145100
20518+99486	110004	120004
68540+11493	070033	080033
96725+23235	129960	119960
58653+91352	140005	150005
88018+94181	182299	182199
73472+66524	149996	139996
56590+43405	109995	099995
68517+85882	154499	154399
72822+97194	160016	170016

Table 6.4: Model errors (addition task, 5 digits, 100k training dataset size).

A similar pattern emerges for 10-digit addition, which also achieved near-perfect accuracy (table 6.1). The failed tasks in table 6.5 show a single digit, typically in the middle, being off by one.

Task	Model result	Correct result
1470939445+3694860228	05165899673	05165799673
9438411468+9410288290	18848799758	18848699758
1600337712+5064162365	06664400077	06664500077
8115628227+5759761714	13875399941	13875389941
2578662692+8570333548	11149996240	11148996240
8757570742+5604529187	14362199929	14362099929
4847761430+3799620568	08647382998	08647381998
7560389844+5150090143	12710489987	12710479987
4277581356+7505693643	11783275999	11783274999
7691931213+8322868827	16014700040	16014800040

Table 6.5: Model errors (addition task, 10 digits, 100k training dataset size).

For 3-digit multiplication, the model achieved an accuracy of about 20% (table 6.2). The errors are affecting only single digits, however some of the digits are off by more than one (table 6.6).

Task	Model result	Correct result
896*551	493096	493696
219*107	023933	023433
192*926	177992	177792
696*135	093060	093960
787*809	636083	636683
884*463	409392	409292
131*907	118617	118817
116*454	052464	052664
763*738	563194	563094
595*730	435350	434350

Table 6.6: Model errors (multiplication task, 3 digits, 100k training dataset size).

For 5-digit multiplication, for which the model did not achieve above-zero accuracy (table 6.2),

the first and last digits were still correct for failed tasks, but multiple digits in the middle of the product were wrong (table 6.7).

Task	Model result	Correct result
67767*25315	1717767705	1715521605
10981*21880	0231011280	0240264280
86313*14577	1251111101	1258184601
98023*21001	2056666623	2058581023
22900*56482	1295185800	1293437800
41039*12284	0501111176	0504123076
14038*12076	0164444688	0169522888
55362*94976	5251717112	5258061312
91694*73471	6731711774	6736849874
90716*70758	6417716608	6418882728

Table 6.7: Model errors (multiplication task, 5 digits, 100k training dataset size).

For 6-digit integer square root, the model achieved about 99% accuracy (table 6.3). The observed errors show the model answer being off by one from the correct answer (table 6.8). Also, the real square root of the failed tasks was very close to an integer for 9 out of 10 failed tasks, which means the task was close to a square number, which are points from which on the integer square root increases by one. It is reasonable to assume that for such a "close call" it is harder to find the exact integer square root.

Task	Model result	Correct result	$\sqrt{Task}$
178980	422	423	423.06028
458306	677	676	676.98301
213411	462	461	461.96428
139840	374	373	373.95187
101789	318	319	319.04388
128218	357	358	358.07541
911019	953	954	954.47315
451595	671	672	672.00818
777861	882	881	881.96428
254001	504	503	503.98512

Table 6.8: Model errors (square root task, 6 digits, 100k training dataset size).

For 20-digit integer square root, the model did not achieve above-zero accuracy (table 6.3). Analyzing some of the failed tasks shows the model predictions only have the leftmost digits correct (table 6.9), with about 4 out of 10 digits of the result being correct. Also, these tasks are not particularly close to a square number, which stands in contrast to the failed tasks for 6 operand digits.

Task	Model result	Correct result	$\sqrt{Task}$
79694857207412785081	8927155525	8927197612	8927197612.20803
10138193304849869414	3183999089	3184052968	3184052968.28584
20740203138915481021	4554099999	4554141317	4554141317.40721
22785206268196001170	4773433272	4773385200	4773385200.06462
61032565798178795501	7812323248	7812334209	7812334209.32430
10321490749937914057	3212499999	3212707697	3212707697.55636
24550288483602635175	4954848488	4954824768	4954824768.20348
95746296144872950099	9785048434	9785003635	9785003635.40418
76700026808202922573	8757999083	8757855148	8757855148.84797
30130695889482657119	5489032224	5489143456	5489143456.81388

Table 6.9: Model errors (square root task, 20 digits, 100k training dataset size).

To conclude, we can see that the errors our model makes follow clear patterns: If the model is trained successfully (achieves good accuracy) the wrong answer is similar to the correct answer, with either a single digit being wrong in the case of addition and multiplication, or the square root being off by one in the case of integer square root. If the model is trained unsuccessfully (zero accuracy achieved) errors often affect more digits (as in the case of 5-digit multiplication).

6.2 Model size experiments

For 10-digit addition, we can see in table 6.10 that all of the tried model configurations were able to learn the task successfully, although there are small differences in final accuracy, such as the $n_{embd} = 768$ model finishing with perfect accuracy on the test dataset. For 5-digit multiplication, we can see that none of the model size changes made the model capable of learning this harder task.

	add (10 digits, 100k)	**mul (5 digits, 100k)**
baseline	0.990	0.000
$n_{embd} = 192$	0.979	0.000
$n_{embd} = 768$	1.000	0.000
$n_{head} = 3$	0.995	0.000
$n_{head} = 12$	0.995	0.000
$n_{layer} = 3$	0.986	0.000
$n_{layer} = 12$	0.991	0.000

Table 6.10: Final model accuracy on the test dataset for the given operation and change in model configuration.

When looking at the development of accuracy for 10-digit addition in fig. 6.4, we can see that there are substantial differences in how quickly the model converges on the addition task when changing the hidden vector size n_{embd}. For $n_{embd} = 192$, it took about 2500 optimization steps for the model to cross 50% accuracy, for $n_{embd} = 384$ (baseline) it took about 900 steps and for $n_{embd} = 768$ only about 500 steps. So in this case the bigger model with more parameters is able to learn faster than smaller models with fewer parameters.

Increasing or decreasing the number of GPT-2 blocks/layers however only made a minor change in convergence speed, with 12 layers performing actually a bit worse than the baseline 6 layers. Changing the number of attention heads, which does not impact the number of model parameters but only the "partitioning" of data inside the model, also had only a minor impact on convergence speed.

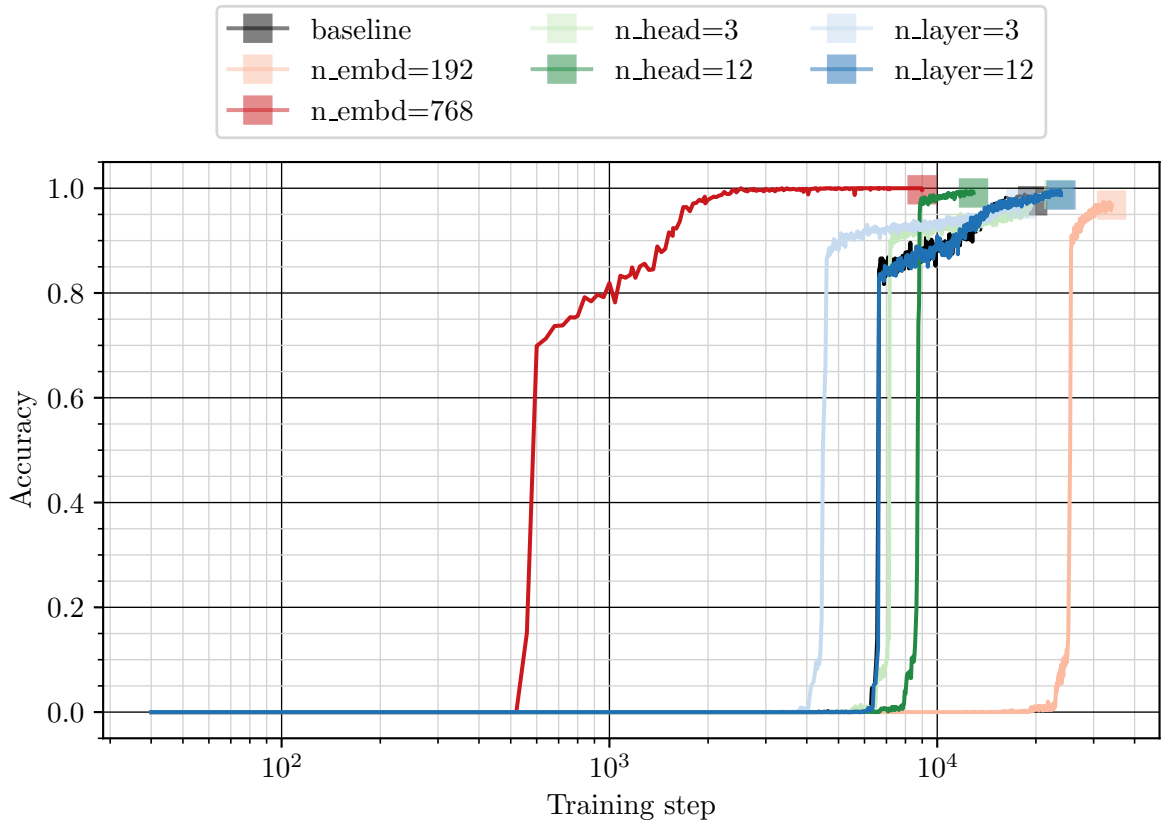

Figure 6.4: Development of accuracy on the validation dataset when training for the addition task (10 digits, 100k dataset) for different changes in model configuration.

6.3 Sampling strategy experiments

For the addition task there were no dramatic differences in achieved model capability for the studied sampling methods (table 6.11 to table 6.14). With a 1k training dataset size, none of the methods achieved high accuracy for any operand size, for 10k it was a mix with high accuracy for 3 operand digits and lower accuracy for 10 operand digits. 100k dataset size allowed for good accuracy for all operand sizes. There were, however, some less significant but still noticeable differences when comparing to the other sampling methods: Uniform digits and uniform bits sampling performed very similar to one another. Both achieved about 40% accuracy for 3 digits, 1k dataset size while basic sampling and from-zero sampling remained close to zero. Here, the mix of easier (smaller, often single-digit) and harder (larger) training samples might have allowed for quicker convergence as we intended. On the other hand, they both had somewhat lower accuracy compared to basic sampling for 10k dataset size. From-zero sampling performed very similar to basic sampling, except for 10 digits, where 10k training samples were not enough to achieve convergence. In that case, it could be that uniform digits and uniform bits sampling did not allow for enough training samples with the full 10 decimal digits in the training dataset if the training dataset size is small and the number of possible bits/digits is large.

	1k	10k	100k
3 digits	0.012	0.999	1.000
5 digits	0.000	0.939	0.997
10 digits	0.000	0.872	0.997

Table 6.11: Addition accuracy, basic sampling.

	1k	10k	100k
3 digits	0.003	0.999	1.000
5 digits	0.000	0.931	0.999
10 digits	0.000	0.000	0.980

Table 6.12: Addition accuracy, from-zero sampling.

	1k	10k	100k
3 digits	0.348	0.931	0.993
5 digits	0.000	0.907	0.987
10 digits	0.000	0.588	0.975

Table 6.13: Addition accuracy, uniform digits sampling.

	1k	10k	100k
3 digits	0.404	0.984	1.000
5 digits	0.007	0.807	0.997
10 digits	0.000	0.566	0.978

Table 6.14: Addition accuracy uniform bits sampling.

As expected, the multiplication task (table 6.15 to table 6.18) is much harder to learn, with most combinations of operand size and dataset size leaving the final model at zero accuracy. This does not change significantly based on the sampling method used, however uniform digits and uniform bits sampling do lead to a slight increase in accuracy for the 100k-dataset, 3-digits case, which given the limited sample space and large number of samples, is the easiest case as expected.

	1k	10k	100k
3 digits	0.000	0.009	0.147
5 digits	0.000	0.000	0.000
10 digits	0.000	0.000	0.000

Table 6.15: Multiplication accuracy, basic sampling.

	1k	10k	100k
3 digits	0.000	0.013	0.223
5 digits	0.000	0.000	0.000
10 digits	0.000	0.000	0.000

Table 6.16: Multiplication accuracy, from-zero sampling.

	1k	10k	100k
3 digits	0.000	0.005	0.354
5 digits	0.000	0.000	0.000
10 digits	0.000	0.000	0.000

Table 6.17: Multiplication accuracy, uniform digits sampling.

	1k	10k	100k
3 digits	0.000	0.009	0.300
5 digits	0.000	0.000	0.002
10 digits	0.000	0.000	0.000

Table 6.18: Multiplication accuracy, uniform bits sampling.

The integer square root task (table 6.19 to table 6.22) is somewhat in between addition and multiplication for learning difficulty. The accuracy tables for basic and from-zero sampling also look very similar for this task. Uniform digits and uniform bits sampling lead to slightly worse accuracy numbers, but the general characteristic is the same: The accuracy numbers mostly depend on the operation learned (addition versus multiplication versus square root) and only slightly change based on the sampling method.

	1k	10k	100k
6 digits	0.139	0.909	0.987
10 digits	0.002	0.043	0.656
20 digits	0.000	0.000	0.000

Table 6.19: Integer square root accuracy, basic sampling.

	1k	10k	100k
6 digits	0.126	0.916	0.983
10 digits	0.004	0.041	0.642
20 digits	0.000	0.000	0.000

Table 6.20: Integer square root accuracy, from-zero sampling.

	1k	10k	100k
6 digits	0.087	0.542	0.895
10 digits	0.000	0.005	0.118
20 digits	0.000	0.000	0.000

Table 6.21: Integer square root accuracy, uniform digits sampling.

	1k	10k	100k
6 digits	0.072	0.427	0.951
10 digits	0.000	0.006	0.135
20 digits	0.000	0.000	0.000

Table 6.22: Integer square root accuracy, uniform bits sampling.

6.4 Result reversal experiments

For the addition task (table 6.24), when compared to the baseline experiments (table 6.23), result reversal led to somewhat successful training (71% accuracy) for the 3 digits, 1k dataset scenario, which could not be learned in baseline. However, training failed for the 10 digits, 10k dataset scenario. To us it is not clear whether this is due to a true difference in the learnability between the two training sample structures, or simply due to variance between training runs. Similar differences observed in the sampling strategy experiments (section 6.3) might point towards the latter. It is also worth noting that for result reversal, 5 of the 9 scenarios, including all operand sizes for the 100k dataset size, led to perfect test accuracy (all 1000 samples correct), while for the baseline addition experiments accuracy was not quite perfect for the experiments with 5 and 10 operand digits.

	1k	10k	100k
3 digits	0.008	1.000	1.000
5 digits	0.000	0.960	0.997
10 digits	0.000	0.826	0.990

Table 6.23: Baseline addition accuracy (without result reversal).

	1k	10k	100k
3 digits	0.709	1.000	1.000
5 digits	0.000	1.000	1.000
10 digits	0.000	0.000	1.000

Table 6.24: Addition accuracy when using result reversal.

For the multiplication task (table 6.26), results were in general very similar to those obtained without result reversal (table 6.25). This operation thus remained the hardest of the three studied and could not be made easier for the model by first presenting the digit-reversed product in the training samples.

	1k	10k	100k
3 digits	0.000	0.013	0.218
5 digits	0.000	0.000	0.000
10 digits	0.000	0.000	0.000

Table 6.25: Baseline multiplication accuracy (without result reversal).

	1k	10k	100k
3 digits	0.001	0.053	0.295
5 digits	0.000	0.000	0.000
10 digits	0.000	0.000	0.000

Table 6.26: Multiplication accuracy when using result reversal.

For the integer square root task (table 6.28), the results were also very similar to the baseline task without result reversal (table 6.27), without significant differences in which scenarios could be learned and the final accuracy numbers obtained.

	1k	10k	100k
6 digits	0.129	0.901	0.983
10 digits	0.004	0.051	0.638
20 digits	0.000	0.000	0.000

Table 6.27: Baseline integer square root accuracy (without result reversal).

	1k	10k	100k
6 digits	0.127	0.921	0.981
10 digits	0.003	0.071	0.839
20 digits	0.000	0.000	0.000

Table 6.28: Integer square root accuracy when using result reversal.

6.4.1 Accuracy development

We also generated accuracy development plots for the result reversal experiments. These plots were combined for the result reversal and scratchpad experiments and are analyzed in section 6.5.1.

6.4.2 Error analysis

We wanted to investigate whether there was any difference in the error patterns between the baseline models and the ones trained using result reversal. To do this, we used the same tasks as for the baseline error analysis (section 6.1.1): addition with 5 and 10 digits (100k dataset), multiplication with 3 and 5 digits (100k dataset) and square root with 6 and 20 digits (100k dataset).

For both 5-digit addition and 10-digit addition, when training using result reversal on a 100k training dataset, there were **no errors** in the entire 100k error analysis dataset, meaning the model performed each addition correctly. This corresponds to the perfect scores seen on the test dataset for these tasks (table 6.24). This stands in contrast to the baseline experiments, where despite 99% accuracy, it was not hard to still find numbers where the addition was performed incorrectly (table 6.4, table 6.5).

For 3-digit multiplication (table 6.29), the error patterns observed were basically the same as for the baseline experiments (table 6.6): Single digits being off by one or, sometimes, more than one. The mistakes are already present in the reversed result, which is then reversed correctly.

Task	Model result	Correct result
896*551=	[692394]493296	[696394]493696
219*107=	[336320]023633	[334320]023433
192*926=	[293771]177392	[297771]177792
696*135=	[060390]093060	[069390]093960
787*809=	[382636]636283	[386636]636683

Table 6.29: Model errors (multiplication task, result reversal, 3 digits, 100k training dataset size).

For 5-digit multiplication (table 6.30), we can also observe similar error patterns compared to the corresponding baseline experiment (table 6.7): The first and last digits of the result are correct, while the digits in the middle are wrong. There also seems to be a pattern of repetition of single digits in the middle of the results, where the model has not learned to output the correct digits.

Task	Model result	Correct result
67767*25315=	[5062665171]1715662605	[5061255171]1715521605
10981*21880=	[0826666320]0236666280	[0824620420]0240264280
86313*14577=	[1059999421]1249999501	[1064818521]1258184601
98023*21001=	[3211119502]2059111123	[3201858502]2058581023
22900*56482=	[0087774921]1294777800	[0087343921]1293437800

Table 6.30: Model errors (multiplication task, result reversal, 5 digits, 100k training dataset size).

For 6-digit integer square root (table 6.31), the error patterns are also similar to the baseline (table 6.8): Off-by-one errors on numbers where the operand is close to a square number, and the model "picked the wrong side".

Task	Model result	Correct result	$\sqrt{Task}$
458306:	[776]677	[676]676	676.98301
478802:	[296]692	[196]691	691.95520
988048:	[399]993	[499]994	994.00604
651341:	[608]806	[708]807	807.05700
942826:	[179]971	[079]970	970.99228

Table 6.31: Model errors (square root task, result reversal, 6 digits, 100k training dataset size).

For 20-digit integer square root (table 6.32), the error patterns once again more or less match the baseline (table 6.9): the 3 most significant digits are correct, the rest are wrong. Compared to the baseline, we can see the result reversal model only got 3 out of 10 most significant digits correct for the examples in the table, whereas for the baseline it was 4 or even 5 correct digits for the examples. Also there is an interesting digit pattern not seen in the baseline errors. A lot of repeated "8"s in the less significant digits where the model is wrong. This corresponds to similar digit repetitions for the multiplication task errors (table 6.30).

Task	Model result	Correct result	$\sqrt{Task}$
79694857207412785081:	[8828828298]8928288288	[2167917298]8927197612	8927197612.208
10138193304849869414:	[8888888813]3188888888	[8692504813]3184052968	3184052968.286
20740203138915481021:	[8828828554]4558288288	[7131414554]4554141317	4554141317.407
22785206268196001170:	[8288282774]4772828828	[0025833774]4773385200	4773385200.065
61032565798178795501:	[8818888187]7818888188	[9024332187]7812334209	7812334209.324

Table 6.32: Model errors (square root task, result reversal, 20 digits, 100k training dataset size).

6.5 Scratchpad experiments

For the addition task (table 6.34), using a scratchpad brings clear improvements over baseline (table 6.33): For the 1k dataset size, training was more or less successful for all operand sizes (good accuracy but not 100%), whereas there was no accuracy for the baseline experiments at 1k dataset size. For the 10k and 100k dataset sizes, accuracy was perfect for all operand sizes, whereas in the baseline experiments accuracy was perfect only for 3-digit addition.

	1k	10k	100k
3 digits	0.008	1.000	1.000
5 digits	0.000	0.960	0.997
10 digits	0.000	0.826	0.990

Table 6.33: Baseline addition accuracy (without scratchpad).

	1k	10k	100k
3 digits	0.992	1.000	1.000
5 digits	0.974	1.000	1.000
10 digits	0.850	1.000	1.000

Table 6.34: Addition accuracy when using scratchpad.

For the multiplication task (table 6.36), there were even more significant improvements made by providing a scratchpad: For the baseline experiments (table 6.35) only 3 digits, 10k dataset and 3 digits, 100k dataset provided above-zero accuracy, and even there it was only about 20%. When using the scratchpad, accuracy is good for 3 digits, 10k and 100k dataset sizes. Even for 5- and 10-digit multiplication, there is some degree of accuracy obtained, whereas it was a flat zero in the baseline experiments.

Because the accuracy numbers for 5-digit and 10-digit multiplication are somewhat "in the middle", showing the model is able to do a significant portion of multiplications correctly, but still making some errors, it is not unreasonable to assume training for longer would have made the model able to perform 5- and 10-digit multiplications with high accuracy. In contrast to the baseline, we had to limit training epochs for the multiplication scratchpad experiments to keep training times reasonable (section 5.8, table 5.3).

It is also an interesting observation that accuracy numbers obtained on the 100k training dataset were lower than those on the 10k dataset. Training was limited to 50 epochs for the 10k dataset and 5 epochs for the 100k dataset (table 5.4). However, the total amount of training batches / optimization steps is the same for both scenarios, but the variety of training samples is lower in the 10k dataset scenario, increasing the risk of overfitting. We would thus have expected better scores for the 100k dataset despite the lower epoch count.

	1k	10k	100k
3 digits	0.000	0.013	0.218
5 digits	0.000	0.000	0.000
10 digits	0.000	0.000	0.000

Table 6.35: Baseline multiplication accuracy (without scratchpad).

	1k	10k	100k
3 digits	0.001	0.941	0.926
5 digits	0.000	0.629	0.487
10 digits	0.000	0.221	0.142

Table 6.36: Multiplication accuracy when using scratchpad.

For the integer square root task (table 6.38) the results were, interestingly, generally worse than baseline (table 6.37). This might be for a number of reasons: First of all, we had to reduce the amount of training epochs, even more so than for the multiplication tasks, due to the very long scratchpads (table 5.2). Thus the resulting models are most likely severely undertrained.

Also, there is an important difference in the scratchpad structure between addition/multiplication and integer square root: For the addition and multiplication operations, the task was decomposed down into the smallest of pieces (single-digit addition and multiplication) (section 4.6.1, section 4.6.2). Whereas for the square root task, we modeled the steps of the binary search algorithm but just left the squaring operations of large numbers as-is, without any explanation on how to compute the squares (we could not include this information as it would have made the scratchpads too massive) (section 4.6.3).

	1k	10k	100k
6 digits	0.129	0.901	0.983
10 digits	0.004	0.051	0.638
20 digits	0.000	0.000	0.000

Table 6.37: Baseline integer square root accuracy (without scratchpad).

	1k	10k	100k
6 digits	0.207	0.738	0.800
10 digits	0.000	0.012	0.006
20 digits	0.000	0.000	0.000

Table 6.38: Integer square root accuracy when using scratchpad.

6.5.1 Accuracy development

We wanted to investigate how quickly the model achieved good accuracy for the various tasks and operand sizes when using result reversal or scratchpad compared to baseline. For this, we plotted the accuracy on the validation dataset during training. The diagrams only show accuracy development for the 100k dataset size for brevity. For scratchpad-based training, we evaluated accuracy only once every epoch and reduced the validation dataset to 10 samples (section 5.8). This of course leads to less precise curves, but can still give a rough idea about training progress.

For the addition task (fig. 6.5) we can see that baseline training needs between 2000 (3 digits) and over 20000 (10 digits) training steps to achieve near 100% accuracy. In contrast, when using result reversal, the model learned somewhat faster (1000-2000 steps for 3 and 5 digits, 10000 steps for 10 digits) and accuracy hits 100% much more abruptly compared to baseline, where it hits 80% and then needs some time to get to 100%. Using a scratchpad leads to much faster convergence, where 3-digit addition already converged at the first log interval at 400 steps, and 5- and 10-digit addition converged before 1000 steps.

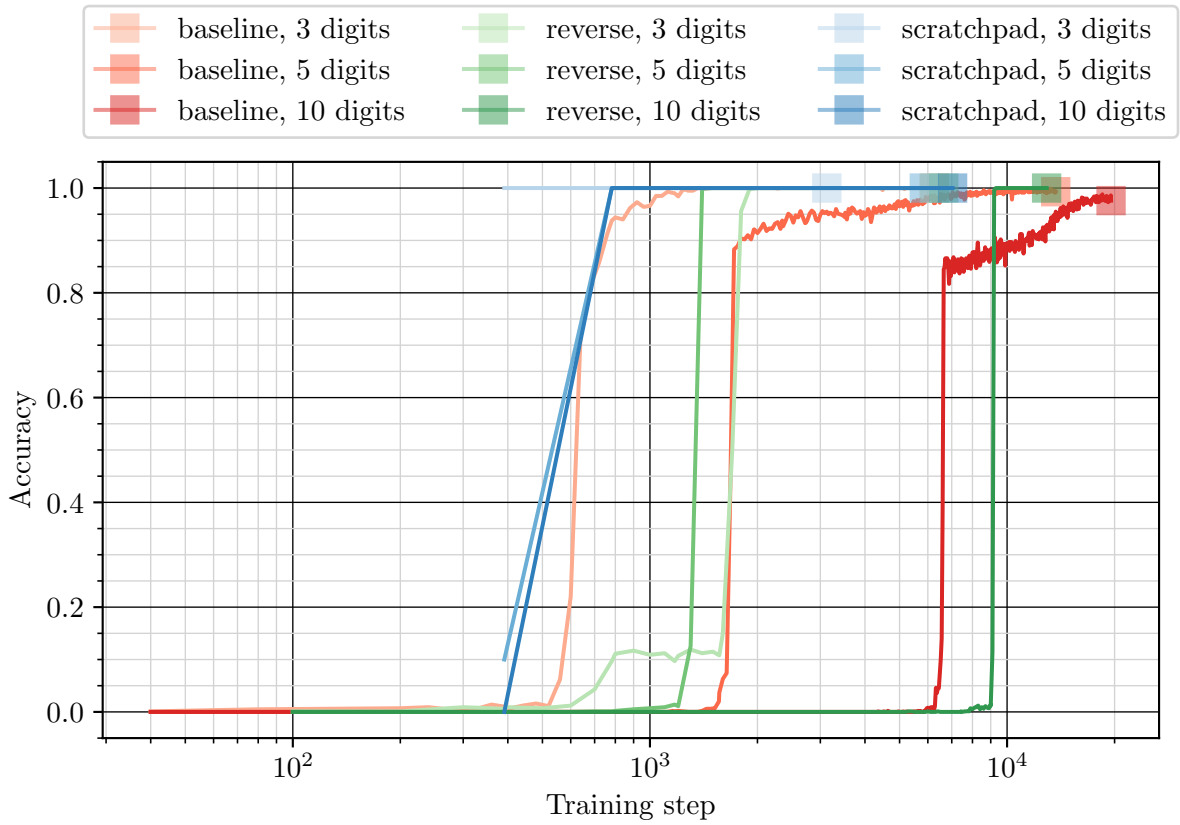

Figure 6.5: Development of accuracy on the validation dataset, when training the addition task using a 100k dataset using various training methods and operand digits.

For the multiplication task (fig. 6.6), we can see that only scratchpad with 3-digit multiplication converged to 100%. For the same task, baseline and result reversal had similar curves, starting to leave zero at about 3000 steps, and ending at around 20% accuracy after about 40000 steps. For 5- and 10-digit multiplication, only scratchpad-based training achieved above-zero accuracy, while still remaining at around 20% accuracy and taking between 3000 and 20000 steps to start converging. It might however converge all the way to 100% given continued training, but resource constraints made us limit the number of training epochs (table 5.4).

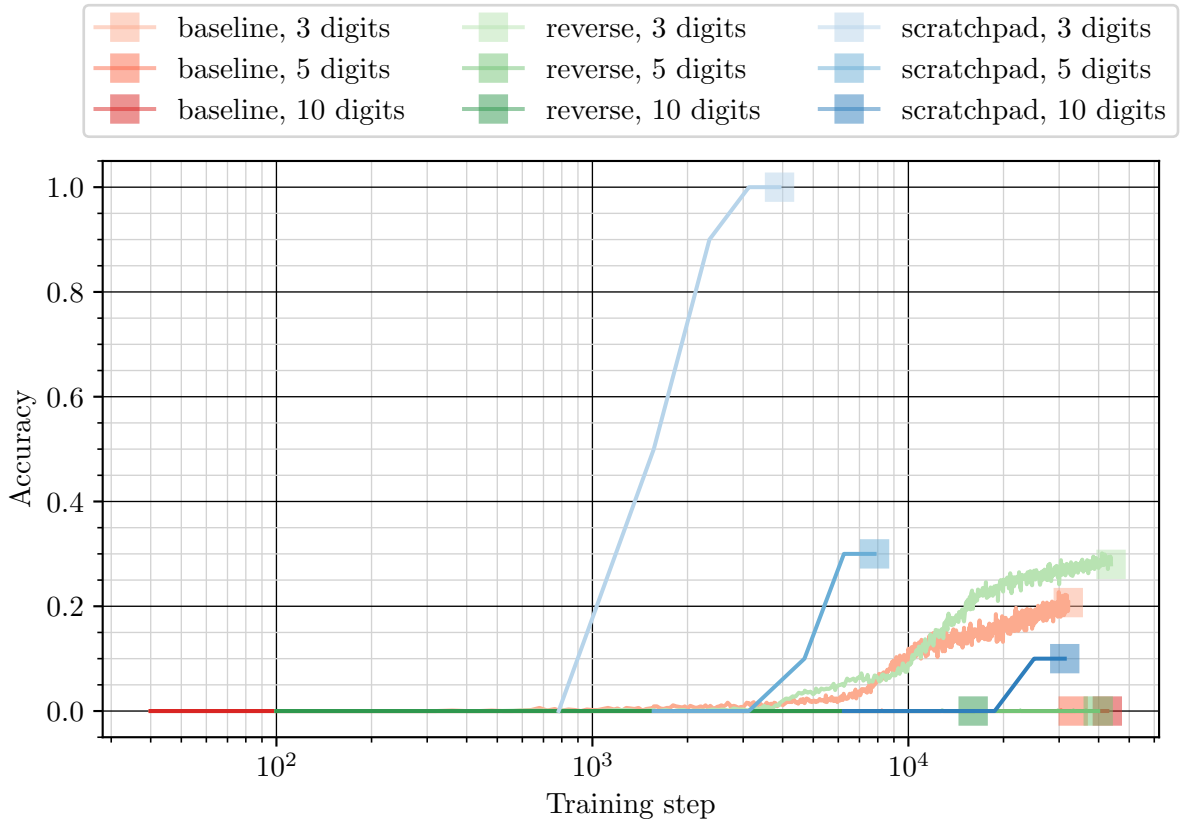

Figure 6.6: Development of accuracy on the validation dataset, when training the multiplication task using a 100k dataset using various training methods and operand digits.

For the square root task with 6 operand digits (fig. 6.7), baseline and result reversal performed very similar while the scratchpad took longer to start converging but then quickly converged, matching the other two training methods. For 10 operand digits, scratchpad actually failed to converge, as opposed to baseline and result reversal. This happened despite taking more optimization steps until training was stopped. It would thus be reasonable to assume that the form of scratchpad we used here, which does not further elaborate on the squaring step, is not helpful for model training, as opposed to the successful addition and multiplication scratchpads.

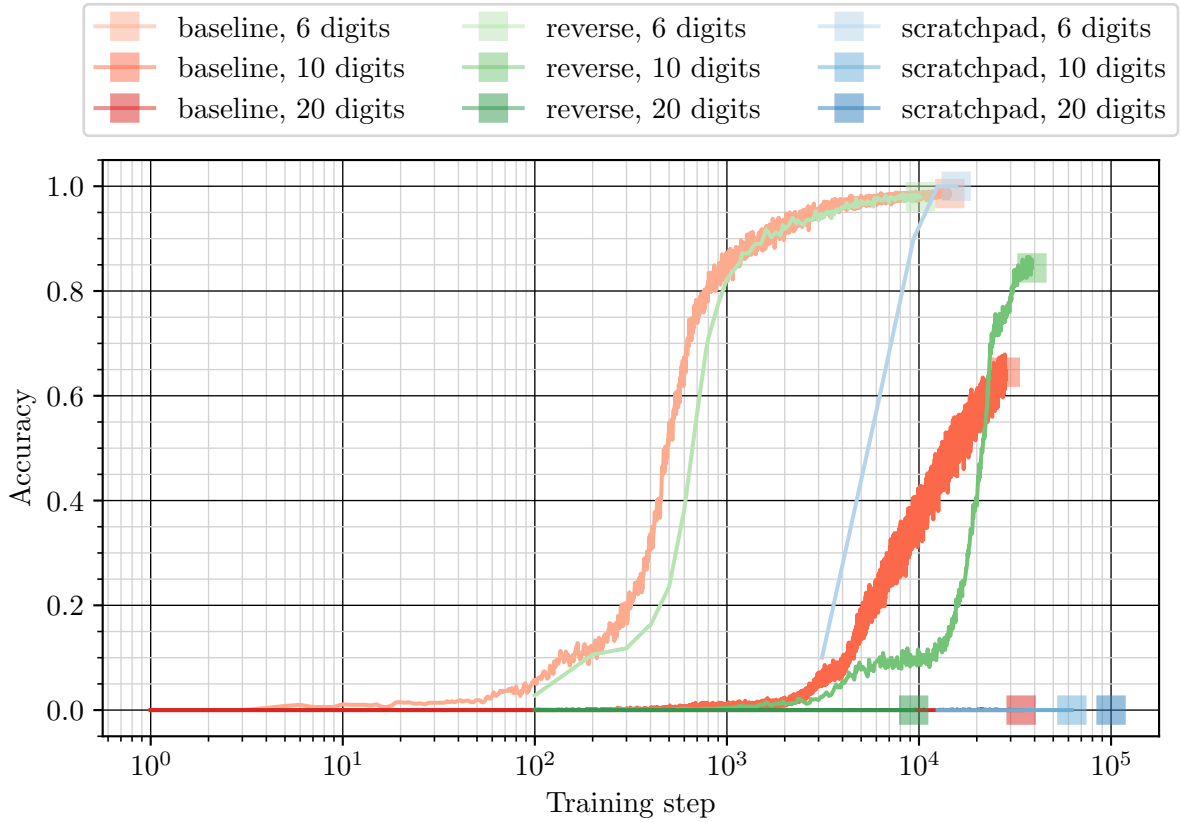

Figure 6.7: Development of accuracy on the validation dataset, when training the square root task using a 100k dataset using various training methods and operand digits.

6.5.2 Error analysis

Similar to the baseline and result reversal experiments, we looked for errors in the model output for the addition task with 5 and 10 digits (100k dataset), multiplication task with 3 and 5 digits (100k dataset) and square root task with 6 digits (100k dataset). We did not analyze errors for the square root task with 20 operand digits, because the scratchpads are so long that presenting them here is impractical. We only looked at three errors per task due to the longer output size when using the scratchpad. The error samples presented are simply the first three samples from the error analysis dataset where the model output did not match the correct output (considering both the scratchpad and final result).

For 5-digit and 10-digit addition using a 100k training dataset, there were no errors in the 100k error analysis dataset, similar to training for addition using result reversal (section 6.4.2). Thus both result reversal and scratchpads allow for training a model that has a very low error probability for addition tasks up to 10 digits.

For 3-digit multiplication, the three error samples in table 6.39 show that there were no errors in the single-digit multiplications. In the first error sample, the model fetched the wrong digit from the single-digit results during the addition phase. We hypothesize this could be due to there being multiple single-digit products that are zero, thus making it harder to pick the right digits due to the uneven length of the products. For the second and third error sample, the model did the entire calculation correctly, but surprisingly failed to read the correct digit sequence from the addition scratchpad.

Model output	Correct output
185*200=[*200{52001,82171,12130}37000, *0{50000,80000,10000}0, *0{50000,80000,10000}0 |{ 000,0,0,0; 0,0,0,0; 0,0,0,0; 3,0,3,0; 3,0,3,0 }] 33000	185*200=[*200{52001,82171,12130}37000, *0{50000,80000,10000}0, *0{50000,80000,10000}0 |{ 000,0,0,0; 0,0,0,0; 0,0,0,0; 7,0,7,0; 3,0,3,0 }] 37000
595*730=[*700{57053,97366,57614}416500, *30{53051,93182,53271}17850, *0{50000,90000,50000}0 |{ 000,0,0,0; 05,0,5,0; 58,0,3,1; 67,1,4,1; 11,1,3,0; 4,0,4,0 }] 454350	595*730=[*700{57053,97366,57614}416500, *30{53051,93182,53271}17850, *0{50000,90000,50000}0 |{ 000,0,0,0; 05,0,5,0; 58,0,3,1; 67,1,4,1; 11,1,3,0; 4,0,4,0 }] 434350
577*170=[*100{71070,71070,51050}57700, *70{77094,77435,57504}40390, *0{70000,70000,50000}0 |{ 000,0,0,0; 09,0,9,0; 73,0,0,1; 70,1,8,0; 54,0,9,0 }] 98080	577*170=[*100{71070,71070,51050}57700, *70{77094,77435,57504}40390, *0{70000,70000,50000}0 |{ 000,0,0,0; 09,0,9,0; 73,0,0,1; 70,1,8,0; 54,0,9,0 }] 98090

Table 6.39: Error samples for the multiplication task (3 digits, 100k training dataset, using scratchpad, trained for 5 epochs).

For 5-digit multiplication, the first error sample (table 6.40) shows the model failing to compute the correct sum based on the correct list of digits and current carry value. This led to some digits in the middle of the product being wrong.

Model output	**Correct output**
67767*25315=[*20000{72041,62131,72151,72151,62131} 1355340000, *5000{75053,65333,75383,75383,65333} 338835000, *300{73012,63202,73232,73232,63202} 20330100, *10{71070,61060,71070,71070,61060} 677670, *5{75053,65333,75383,75383,65333} 338835 |{ 00005,0,5,0; 00073,0,0,1; 00168,1,6,1; 05078,1,1,2; 43373,2,3,2; 38363,2,7,2; 580,2,5,1; 532,1,1,1; 33,1,7,0; 1,0,1,0 }] 1715731605	67767*25315=[*20000{72041,62131,72151,72151,62131} 1355340000, *5000{75053,65333,75383,75383,65333} 338835000, *300{73012,63202,73232,73232,63202} 20330100, *10{71070,61060,71070,71070,61060} 677670, *5{75053,65333,75383,75383,65333} 338835 |{ 00005,0,5,0; 00073,0,0,1; 00168,1,6,1; 05078,1,1,2; 43373,2,2,2; 38363,2,5,2; 580,2,5,1; 532,1,1,1; 33,1,7,0; 1,0,1,0 }] 1715521605

Table 6.40: First error sample for the multiplication task (5 digits, 100k training dataset, using scratchpad, trained for 5 epochs).

The second error sample (table 6.41) shows the same pattern: wrong sum adding multiple digits during the addition phase - leading to some wrong digits in the middle of the final product.

Model output	**Correct output**
10981*21880=[*20000{12020,82061,92191,02110,12020} 219620000, *1000{11010,81080,91090,01000,11010} 10981000, *800{18080,88046,98687,08770,18080} 8784800, *80{18080,88046,98687,08770,18080} 878480, *0{10000,80000,90000,00000,10000} 0 |{ 00000,0,0,0; 0008,0,8,0; 0084,0,2,1; 0148,1,4,1; 2887,1,8,2; 6978,2,0,3; 908,3,0,2; 11,2,4,0; 2,0,2,0 }] 240084280	10981*21880=[*20000{12020,82061,92191,02110,12020} 219620000, *1000{11010,81080,91090,01000,11010} 10981000, *800{18080,88046,98687,08770,18080} 8784800, *80{18080,88046,98687,08770,18080} 878480, *0{10000,80000,90000,00000,10000} 0 |{ 00000,0,0,0; 0008,0,8,0; 0084,0,2,1; 0148,1,4,1; 2887,1,6,2; 6978,2,2,3; 908,3,0,2; 11,2,4,0; 2,0,2,0 }] 240264280

Table 6.41: Second error sample for the multiplication task (5 digits, 100k training dataset, using scratchpad, trained for 5 epochs).

The third error sample (table 6.42) shows a somewhat different pattern: Interestingly, there were mistakes in a single-digit multiplication with factor 1, basically failing to copy the digits. This led to wrong digits in the middle of the final product. In summary, there was only one error observed in the digit multiplication stage and one error writing the final result, while there were multiple errors in the sum stage, either computing the wrong sum or fetching the wrong digits.

Model output	**Correct output**
98023*21001=[*20000{32060,22040,02000,82061,92191} 1960460000, *1000{31030,21020,01000,81080,91090} 98023000, *0{30000,20000,00000,80000,90000} 0, *0{30000,20000,00000,80000,90000} 0, *1{31030,21020,21020,01000,81080} 80223 |{ 00003,0,3,0; 002,0,2,0; 002,0,2,0; 030,0,3,0; 628,0,6,1; 40,1,5,0; 08,0,8,0; 69,0,5,1; 9,1,0,1; 1,1,2,0 }] 2058563223	98023*21001=[*20000{32060,22040,02000,82061,92191} 1960460000, *1000{31030,21020,01000,81080,91090} 98023000, *0{30000,20000,00000,80000,90000} 0, *0{30000,20000,00000,80000,90000} 0, *1{31030,21020,01000,81080,91090} 98023 |{ 00003,0,3,0; 002,0,2,0; 000,0,0,0; 038,0,1,1; 629,1,8,1; 40,1,5,0; 08,0,8,0; 69,0,5,1; 9,1,0,1; 1,1,2,0 }] 2058581023

Table 6.42: Third error sample for the multiplication task (5 digits, 100k training dataset, using scratchpad, trained for 5 epochs).

For the square root task with 6 digits, in the first error sample (table 6.43) we can see the model making mistakes in squaring large numbers at the beginning of the algorithm. This, however, did not influence the final result as the algorithm always has to take a few steps in the same direction at the start due do the large search interval $[0, n]$. The model did make another mistake at the very end of the algorithm, picking the wrong half of the search space and then picking the wrong one of two numbers for the final square root.

Model output	**Correct output**
915965:[	915965:[
{0,915965}457982*457982=209199148324	{0,915965}457982*457982=209747512324
{0,457982}228991*228991=52299787081	{0,457982}228991*228991=52436878081
{0,228991}114495*114495=13074832025	{0,228991}114495*114495=13109105025
{0,114495}57247*57247=3268650009	{0,114495}57247*57247=3277219009
{0,57247}28623*28623=817133129	{0,57247}28623*28623=819276129
{0,28623}14311*14311=204268721	{0,28623}14311*14311=204804721
{0,14311}7155*7155=51060025	{0,14311}7155*7155=51194025
{0,7155}3577*3577=12761929	{0,7155}3577*3577=12794929
{0,3577}1788*1788=3188944	{0,3577}1788*1788=3196944
{0,1788}894*894=799236	{0,1788}894*894=799236
{894,1788}1341*1341=1798281	{894,1788}1341*1341=1798281
{894,1341}1117*1117=1247689	{894,1341}1117*1117=1247689
{894,1117}1005*1005=1010025	{894,1117}1005*1005=1010025
{894,1005}949*949=900601	{894,1005}949*949=900601
{949,1005}977*977=954529	{949,1005}977*977=954529
{949,977}963*963=927369	{949,977}963*963=927369
{949,963}956*956=913936	{949,963}956*956=913936
{956,963}959*959=919681	{956,963}959*959=919681
{956,959}957*957=915849	{956,959}957*957=915849
{956,957}957*957=915849	{957,959}958*958=917764
{956,957}	{957,958}
]	]
956	957

Table 6.43: First error sample for the square root task (6 digits, 100k training dataset, using scratchpad, trained for 5 epochs).

In the second error sample (table 6.44) we can see the model making similar errors computing large squares at the start. It then did fine computing smaller squares and picking the right half of the search space. At the very end, it picked the wrong half of the search space, similarly to the first example.

Model output	**Correct output**
195046:[	195046:[
{0,195046}97523*97523=9508185529	{0,195046}97523*97523=9510735529
{0,97523}48761*48761=2377097121	{0,97523}48761*48761=2377635121
{0,48761}24380*24380=594244400	{0,48761}24380*24380=594384400
{0,24380}12190*12190=148561600	{0,24380}12190*12190=148596100
{0,12190}6095*6095=37140025	{0,12190}6095*6095=37149025
{0,6095}3047*3047=9281209	{0,6095}3047*3047=9284209
{0,3047}1523*1523=2318529	{0,3047}1523*1523=2319529
{0,1523}761*761=579121	{0,1523}761*761=579121
{0,761}380*380=144400	{0,761}380*380=144400
{380,761}570*570=324900	{380,761}570*570=324900
{380,570}475*475=225625	{380,570}475*475=225625
{380,475}427*427=182329	{380,475}427*427=182329
{427,475}451*451=203401	{427,475}451*451=203401
{427,451}439*439=192721	{427,451}439*439=192721
{439,451}445*445=198025	{439,451}445*445=198025
{439,445}442*442=195364	{439,445}442*442=195364
{439,442}440*440=193600	{439,442}440*440=193600
{440,442}441*441=194481	{440,442}441*441=194481
{440,441}	{441,442}
]	]
440	441

Table 6.44: Second error sample for the square root task (6 digits, 100k training dataset, using scratchpad, trained for 5 epochs).

The third error sample (table 6.45) shows the model coming to the right conclusion after making a few mistakes computing larger squares at the start. In summary, we can see that errors happen when computing larger square numbers at the start of the algorithm and when picking the right half at the end of the algorithm. Interestingly, computing squares towards of the algorithm does not seem a cause of errors.

Model output	**Correct output**
562141:[	562141:[
{0,562141}281070*281070=78175724900	{0,562141}281070*281070=79000344900
{0,281070}140535*140535=19543931225	{0,281070}140535*140535=19750086225
{0,140535}70267*70267=4885917289	{0,140535}70267*70267=4937451289
{0,70267}35133*35133=1221444689	{0,70267}35133*35133=1234327689
{0,35133}17566*17566=305343356	{0,35133}17566*17566=308564356
{0,17566}8783*8783=76335089	{0,17566}8783*8783=77141089
{0,8783}4391*4391=19079881	{0,8783}4391*4391=19280881
{0,4391}2195*2195=4767025	{0,4391}2195*2195=4818025
{0,2195}1097*1097=1190409	{0,2195}1097*1097=1203409
{0,1097}548*548=299304	{0,1097}548*548=300304
{548,1097}822*822=675684	{548,1097}822*822=675684
{548,822}685*685=469225	{548,822}685*685=469225
{685,822}753*753=567009	{685,822}753*753=567009
{685,753}719*719=516961	{685,753}719*719=516961
{719,753}736*736=541696	{719,753}736*736=541696
{736,753}744*744=553536	{736,753}744*744=553536
{744,753}748*748=559504	{744,753}748*748=559504
{748,753}750*750=562500	{748,753}750*750=562500
{748,750}749*749=561001	{748,750}749*749=561001
{749,750}	{749,750}
]	]
749	749

Table 6.45: Third error sample for the square root task (6 digits, 100k training dataset, using scratchpad, trained for 5 epochs).

6.6 Attention heatmaps

We generated attention heatmap diagrams for some example tasks processed by the corresponding models as outlined in section 4.7 and section 5.9. These diagrams are to be interpreted as follows:

- Each row refers to the sequence position where a certain token is generated by the model (the position in the token sequence where the model outputs this token based on the previous ones as input).
- For each row, the columns of that row refer to previous token positions that the model will consider for generating the output at that position. Note that the attention values for the current and following token positions are always zero, as the model has not yet generated these values at that point.
- Colors show attention intensity and range from dark purple (no attention to this position) to yellow (full attention to this position).

6.6.1 Addition task

Attention activation for the addition task "123+456=0579" is illustrated in fig. 6.8 for each of the six GPT-2 blocks and each of the six attention heads per block. When looking for patterns in these activation heatmaps, a clear relationship is visible in Block 1, where the position of the output digit "5" is mostly attending to the positions to the input digits "1" and "4", the position of the output digit "7" to the input digits "2" and "5" and the output digit "9" to the input digits "3" and "6". The first output digit "0" also attends mostly to the input digits "1" and "4". This is consistent with how data would flow from input to output in a standard addition algorithm. The stop token "." position does not attend to any input position in particular. This pattern is more or less the same across all six attention heads. When looking at the later blocks, there are also some spots of high activation, however these are often not uniform across attention heads and there is no obvious pattern between input and outputs.

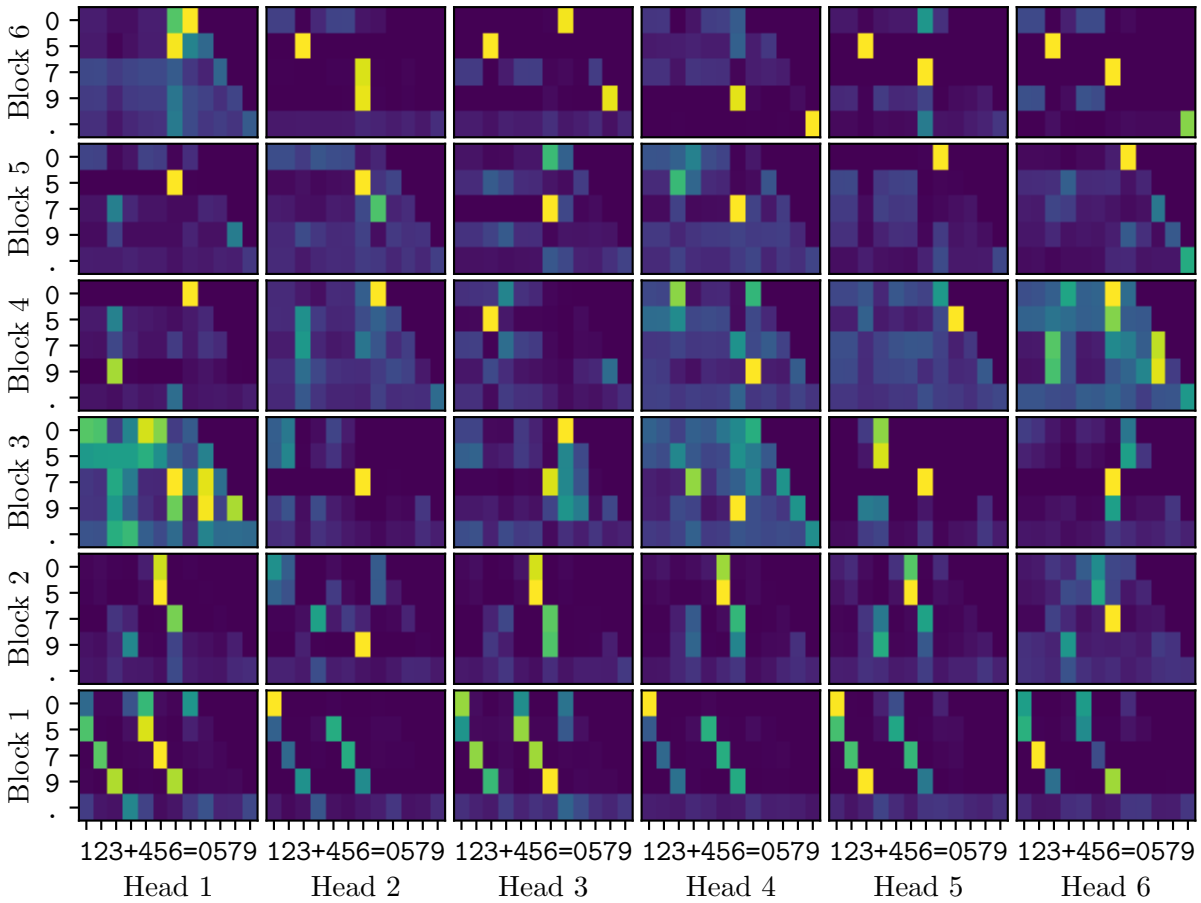

Figure 6.8: Attention activation heatmap for all blocks and attention heads for the addition task "123+456".

Since we found out that the most visible attention patterns are in the first block, to keep the diagrams of reasonable size especially for larger sequences, for the rest of the diagrams only the attention mechanism of the first GPT-2 block is considered, and the attention values of all six heads are averaged. The single-diagram heatmap of fig. 6.9 for the same addition task shows the same pattern mentioned before, with an output digit being connected to those two input digits being added for this position, with the relationship being clear for the digits "579" and a bit less prominent for the digit "0".

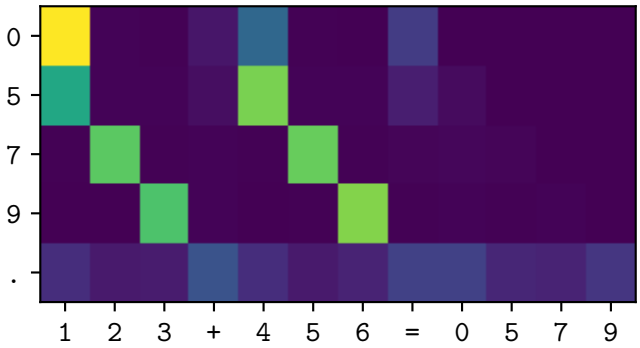

Figure 6.9: First block attention heatmap for the addition task "123+456", averaged over all six heads.

There is little change to the previous diagram in fig. 6.10 which shows the same addition model working on different numbers, namely the addition "999+001". Only minor changes in attention value are visible when doing addition of different numbers using the same model.

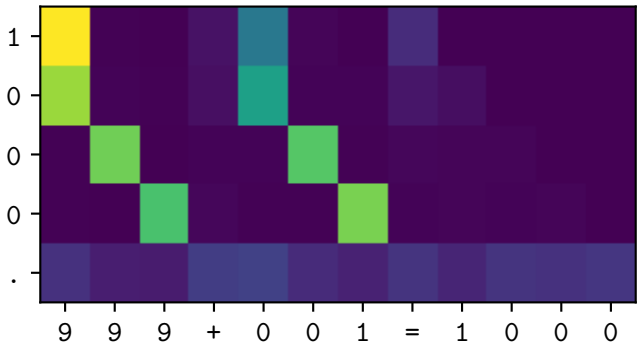

Figure 6.10: First block attention heatmap for the addition task "999+001", averaged over all six heads.

When switching to a model trained for 5-digit addition and the addition task "12345+67890" (fig. 6.11) we can still see a similar pattern of output digits referring to those input digits that make sense when thinking about an addition algorithm. However the pattern is somewhat more diffuse for the more significant output digits, which are also attending to less significant digits of the input numbers somewhat.

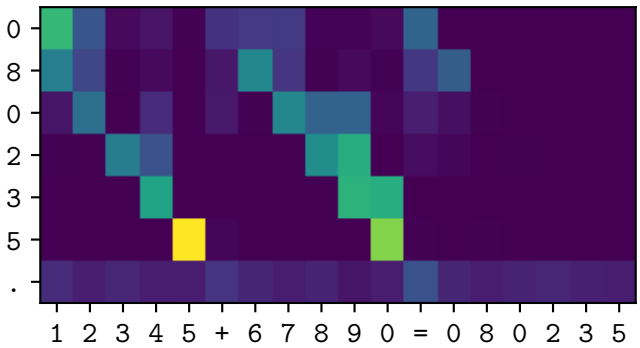

Figure 6.11: First block attention heatmap for the addition task "12345+67890", averaged over all six heads.

For 3-digit addition using result reversal (fig. 6.12), we can see a very clear pattern of the final output digits attending to the corresponding digits of the reversed result. There however is not such a clear pattern for the reversed addition that happens before, with the output digits "9750" not having a strong connection to the matching input digits as was seen before for normal addition.

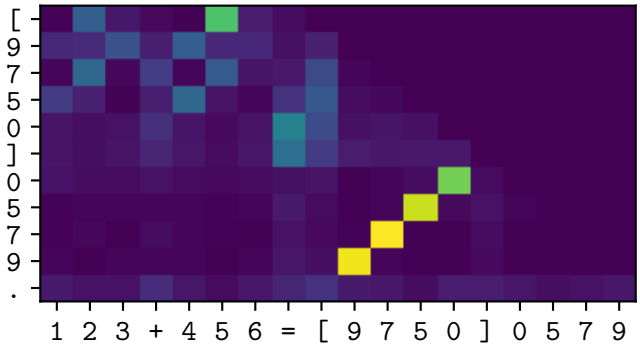

Figure 6.12: First block attention heatmap for the addition task "123+456", using result reversal, averaged over all six heads.

For the addition task using the scratchpad (fig. 6.13), we can also see some patterns: For the individual addition steps, the input digits are fetched as the first two digits of each step. We can also see that the old carry value (third digit of each step) to some degree attends to the fifth digit of the previous step, which is the output carry value of that previous step.

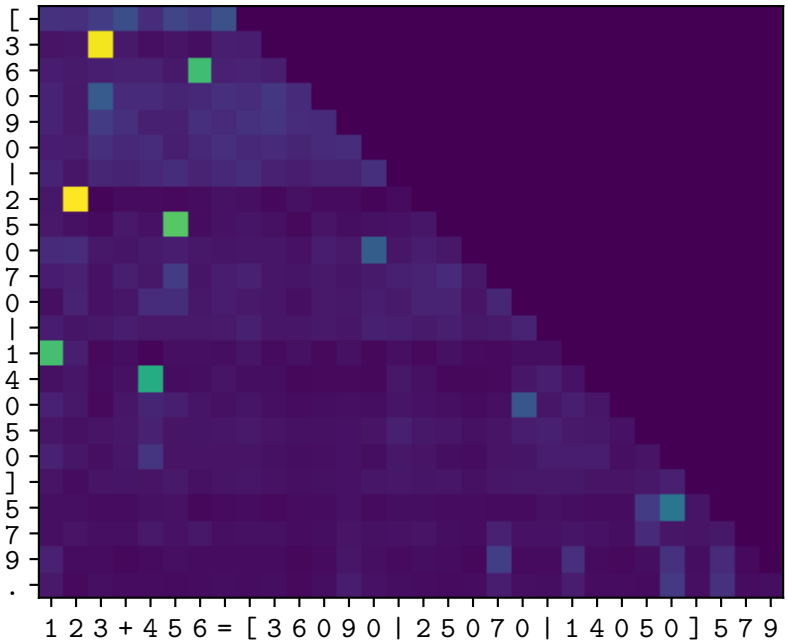

Figure 6.13: First block attention heatmap for the addition task "123+456", using scratchpad, averaged over all six heads.

6.6.2 Multiplication task

For the multiplication task with 3 digits (fig. 6.14) our model actually computes the wrong result 056188 instead of the correct result 056088 which is not unexpected considering its low accuracy at about 20% (table 6.2).

We can see the 3 least significant digits of the product (188) attending primarily to the least significant digits of the input numbers (3 and 6). The 3rd most significant digit (6) has an attention pattern without clear focus. The 1st and 2nd most significant digits of the product (0 and 5) attend to the most significant digits of the input (1 and 4).

This does not follow a pattern clearly related to a normal multiplication algorithm: For the least significant digit of the product only the least significant digit of the factors should be relevant, for the second least significant digit this should extend to the first and second digit of the factors and so on, which does not match the pattern visible in the diagram. This corresponds well to the fact that accuracy achieved for multiplication was much poorer that for addition (only 22% for 3-digit multiplication, see table 6.2), prompting speculation whether models with high accuracy on an arithmetic task show more meaningful digit relationships in their heatmaps that those with low accuracy.

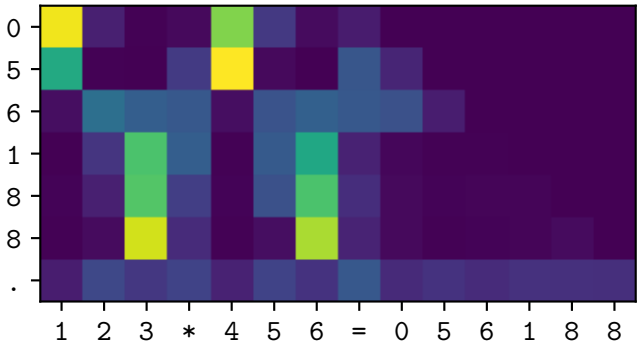

Figure 6.14: First block attention heatmap for the multiplication task "123*456", averaged over all six heads.

6.6.3 Square root task

For the square root tasks with 6 digits (fig. 6.15) we can see the least significant digit of the output "1" attending to the 3 most significant digits of the input (and slightly to previous output digit "3") whereas the more significant output digit 5 attends mostly to the second most significant input digit "2" and the most significant output digit "3" attends mostly to the most significant input digit "1".

This also does not clearly correspond to a normal integer square root algorithm as for example the least significant 3 digits of the input are not considered at all by the model's attention pattern, while for example 123000 has an integer square root of 350 whereas 123999 has a square root of 352. These digits must thus be considered, if they are even considered at all, only at later blocks of the model not visible in this diagram.

Accuracy was 98% for 6-digit integer square root (table 6.3), but for this operation it is plausible that the model just guesses the square root based on the first few digits, which would of course fail on numbers close to a square number which require precise computation, but such hard cases should be only a small fraction of the test dataset (which is uniformly selected from all 6-digit integers).

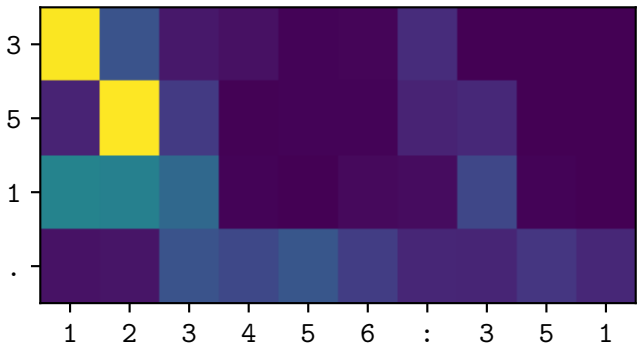

Figure 6.15: First block attention heatmap for the integer square root task "123456", averaged over all six heads.

7 Summary

In this work, we investigated the performance of small transformer models on the arithmetic operations of addition, multiplication and integer square root. To establish a baseline, we trained transformer networks with GPT-2 architecture and about 10 million parameters on each combination of these three operations and various operand lengths and training dataset sizes per operation.

We then wanted to improve performance, in particular for the multiplication and integer square root tasks and thus attempted a number of changes to the model and dataset. First, we explored whether changing the model size (number of layers, hidden vector sizes etc.) improved model performance (section 6.2). After that, we tried changing the sampling function for the training dataset to see whether prioritizing certain training samples improved performance (section 6.3). Then, we went on to change the actual training samples themselves. For this, we tried two different strategies: The first strategy was reversing the digits of the result for each training sample (section 6.4). The second strategy was augmenting each training sample with a scratchpad of intermediate steps of the arithmetic computation of that sample (section 6.5). Finally, we created heatmaps of model attention to get some insight into how the model processes input strings (section 6.6).

7.1 Results

The best results we could achieve for the arithmetic functions we studied were 100% accuracy for 3-, 5- and 10-digit addition, 94% for 3-digit multiplication, 63% for 5-digit multiplication, 22% for 10-digit multiplication, 98% for 6-digit integer square root, 64% for 10-digit integer square root and 0% for 20-digit integer square root. For addition and multiplication, these results were obtained when using scratchpads, for integer square root, they are from the baseline model. The largest training dataset size (100k training samples) was used.

On the one hand, these results could likely be made even better by longer training runs when using scratchpads (we constrained epochs to keep training time manageable with the long scratchpad strings). On the other hand, using scratchpads also requires knowledge of meaningful intermediate steps - how to compute the output from the input step by step. A key advantage of machine learning over traditional algorithms is that this knowledge of how to compute the result is generally not required: The model is provided only a sequence of input and output pairs with the optimizer handling how to make the model get from input to output. This advantage is lost when having to devise a step-by-step computation for the training samples.

Other strategies we tried for improving performance such as changing the dimensions of the model, dataset sampling strategy, or reversing the digits of the results generally brought only improvements to convergence speed (number of optimization steps until the model learned the task) or improved model accuracy from an already high 98-99% to 100% (no detectable errors). These strategies did not give the desired "quantum leap" of making multiplication or square root with large operands learnable.

7.2 Future work

An obvious avenue for building on our results would be running training on the scratchpad experiments for longer, until an actual plateau is reached. Training using scratchpad was limited by the resources and budget. Training for more epochs would likely allow to see the actual limitations of scratchpad-aided multiplication on our model better, as we have reason to believe our models were undertrained (section 6.5). It is also possible that the stopping condition we used contributes to premature termination of training; analyzing accuracy development curves suggests this might be a possibility. Focusing not only on loss but also accuracy might ensure training does not stop before the model reaches maximum accuracy.

We did not analyze variances between multiple training runs as even a single training run took substantial resources. It would however be interesting to see if a task that normally fails to converge occasionally does converge or vice versa. It would be worth combining result reversal with uniform-digit sampling to try and replicate the good results for multiplication in [36]. It would also be interesting to see whether a second model could learn to predict where the main model made a mistake and try to correct it, especially for long output sequences when using scratchpads.

The good results of basic feed-forward networks in [6] would make it promising to explore their limits, especially with regards to longer scratchpad sequences. It remains to determine at which point transformers start having an advantage over more basic and established models. Other architectures like one-dimensional CNNs or recurrent networks could be compared too.

Using a simpler, more straightforward scratchpad for multiplication as in [35] would reduce sequence length and thus accelerate training. It is worthwhile to compare performance between such a bare-bones scratchpad and the more elaborate one we are using. Furthermore, a better scratchpad for integer square root should be devised. It must be reasonably short, yet not leave big chunks of computation without intermediate steps, which we identified as a potential problem of our square root scratchpad.

Finally, the inner mechanisms of the model could be explored further. The heatmap diagrams in section 6.6 show clear, logical relationships between input and output in the first layer already - but it does not explain the whole "algorithm" learned by the model. Analysis would have to focus on the more opaque patterns in later layers of the model.

Bibliography

References

[1] Ashish Vaswani et al. "Attention is All you Need." In: *Advances in Neural Information Processing Systems*. Ed. by I. Guyon et al. Vol. 30. Curran Associates, Inc., 2017. URL: https://arxiv.org/pdf/1706.03762.

[2] Alexey Dosovitskiy et al. "An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale." In: *International Conference on Learning Representations*. 2021. URL: https://arxiv.org/pdf/2010.11929.

[3] Jaromir Savelka et al. "Thrilled by Your Progress! Large Language Models (GPT-4) No Longer Struggle to Pass Assessments in Higher Education Programming Courses." In: *Proceedings of the 2023 ACM Conference on International Computing Education Research V.1*. ICER 2023. ACM, Aug. 2023, pp. 78–92. URL: https://arxiv.org/pdf/2306.10073.

[4] Michael Hahn. "Theoretical Limitations of Self-Attention in Neural Sequence Models." In: *Transactions of the Association for Computational Linguistics* 8 (Dec. 2020), pp. 156–171. ISSN: 2307-387X. URL: https://arxiv.org/pdf/1906.06755.

[5] Leonardo Franco and Sergio A. Cannas. "Solving arithmetic problems using feed-forward neural networks." In: *Neurocomputing* 18.1 (1998), pp. 61–79. ISSN: 0925-2312. URL: https://www.sciencedirect.com/science/article/pii/S0925231297000696.

[6] Yedid Hoshen and Shmuel Peleg. "Visual Learning of Arithmetic Operations." In: *Proceedings of the Thirtieth AAAI Conference on Artificial Intelligence, February 12-17, 2016, Phoenix, Arizona, USA*. Ed. by Dale Schuurmans and Michael P. Wellman. AAAI Press, 2016, pp. 3733–3739. URL: https://arxiv.org/pdf/1506.02264.

[7] Nayoung Lee et al. "Teaching Arithmetic to Small Transformers." In: *International Conference on Representation Learning*. Ed. by B. Kim et al. Vol. 2024. 2024, pp. 25001–25054. URL: https://arxiv.org/pdf/2307.03381.

[8] Davide Maltoni and Matteo Ferrara. "Arithmetic with language models: From memorization to computation." In: *Neural Networks* 179 (2024), p. 106550. ISSN: 0893-6080. URL: https://arxiv.org/pdf/2308.01154.

[9] Sean McLeish et al. "Transformers Can Do Arithmetic with the Right Embeddings." In: *Advances in Neural Information Processing Systems*. Ed. by A. Globerson et al. Vol. 37. Curran Associates, Inc., 2024, pp. 108012–108041. URL: https://arxiv.org/pdf/2405.17399.

[10] Hanseul Cho et al. "Arithmetic Transformers Can Length-Generalize in Both Operand Length and Count." In: *International Conference on Representation Learning*. Ed. by Y. Yue et al. Vol. 2025. 2025, pp. 79987–80029. URL: https://arxiv.org/pdf/2410.15787.

[11] Kyunghyun Cho et al. "Learning Phrase Representations using RNN Encoder-Decoder for Statistical Machine Translation." In: *Proceedings of the 2014 Conference on Empirical Methods in Natural Language Processing, EMNLP 2014, October 25-29, 2014, Doha, Qatar, A meeting of SIGDAT, a Special Interest Group of the ACL*. Ed. by Alessandro Moschitti, Bo Pang, and Walter Daelemans. ACL, 2014, pp. 1724–1734. URL: https://arxiv.org/pdf/1406.1078.

[12] Jonas Gehring et al. "Convolutional Sequence to Sequence Learning." In: *Proceedings of the 34th International Conference on Machine Learning, ICML 2017, Sydney, NSW, Australia, 6-11 August 2017*. Ed. by Doina Precup and Yee Whye Teh. Vol. 70. Proceedings of Machine Learning Research. PMLR, 2017, pp. 1243–1252. URL: https://arxiv.org/pdf/1705.03122.

[13] Ilya Sutskever, Oriol Vinyals, and Quoc V. Le. "Sequence to Sequence Learning with Neural Networks." In: *Advances in Neural Information Processing Systems 27: Annual Conference on Neural Information Processing Systems 2014, December 8-13 2014, Montreal, Quebec, Canada*. Ed. by Zoubin Ghahramani et al. 2014, pp. 3104–3112. URL: https://arxiv.org/pdf/1409.3215.

[14] Jacob Devlin et al. "BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding." In: *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (Long and Short Papers)*. Ed. by Jill Burstein, Christy Doran, and Thamar Solorio. Minneapolis, Minnesota: Association for Computational Linguistics, June 2019, pp. 4171–4186. URL: https://aclanthology.org/N19-1423/.

[15] Ming Ding et al. "CogView: Mastering Text-to-Image Generation via Transformers." In: *Advances in Neural Information Processing Systems*. Ed. by M. Ranzato et al. Vol. 34. Curran Associates, Inc., 2021, pp. 19822–19835. URL: https://arxiv.org/pdf/2105.13290.

[16] Anian Ruoss et al. "Amortized Planning with Large-Scale Transformers: A Case Study on Chess." In: *Advances in Neural Information Processing Systems*. Ed. by A. Globerson et al. Vol. 37. Curran Associates, Inc., 2024, pp. 65765–65790. URL: https://arxiv.org/pdf/2402.04494.

[17] Rico Sennrich, Barry Haddow, and Alexandra Birch. "Neural Machine Translation of Rare Words with Subword Units." In: *Proceedings of the 54th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. Ed. by Katrin Erk and Noah A. Smith. Berlin, Germany: Association for Computational Linguistics, Aug. 2016, pp. 1715–1725. URL: https://arxiv.org/pdf/1508.07909.

[18] Mehdi Ali et al. "Tokenizer Choice For LLM Training: Negligible or Crucial?" In: *Findings of the Association for Computational Linguistics: NAACL 2024*. Ed. by Kevin Duh, Helena Gomez, and Steven Bethard. Mexico City, Mexico: Association for Computational Linguistics, June 2024, pp. 3907–3924. URL: https://aclanthology.org/2024.findings-naacl.247/.

[19] Linting Xue et al. "ByT5: Towards a Token-Free Future with Pre-trained Byte-to-Byte Models." In: *Transactions of the Association for Computational Linguistics* 10 (Mar. 2022), pp. 291–306. ISSN: 2307-387X. URL: https://arxiv.org/pdf/2105.13626.

[20] Jianlin Su et al. "RoFormer: Enhanced transformer with Rotary Position Embedding." In: *Neurocomputing* 568 (2024), p. 127063. ISSN: 0925-2312. URL: https://arxiv.org/pdf/2104.09864.

[21] Daphne Ippolito et al. "Comparison of Diverse Decoding Methods from Conditional Language Models." In: *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics*. Ed. by Anna Korhonen, David Traum, and Lluis Marquez. Florence, Italy: Association for Computational Linguistics, July 2019, pp. 3752–3762. URL: https://arxiv.org/pdf/1906.06362.

[22] Sanghyun Yi et al. "Towards Coherent and Engaging Spoken Dialog Response Generation Using Automatic Conversation Evaluators." In: *Proceedings of the 12th International Conference on Natural Language Generation*. Ed. by Kees van Deemter, Chenghua Lin, and Hiroya Takamura. Tokyo, Japan: Association for Computational Linguistics, Oct. 2019, pp. 65–75. URL: https://arxiv.org/pdf/1904.13015.

[23] Long Ouyang et al. "Training language models to follow instructions with human feedback." In: *Advances in Neural Information Processing Systems*. Ed. by S. Koyejo et al. Vol. 35. Curran Associates, Inc., 2022, pp. 27730–27744. URL: https://proceedings.neurips.cc/paper_files/paper/2022/hash/b1efde53be364a73914f58805a001731-Abstract-Conference.html.

[24] Lisa Torrey and Jude Shavlik. "Transfer learning." In: *Handbook of research on machine learning applications and trends: algorithms, methods, and techniques*. IGI global, 2010, pp. 242–264.

[25] Ofir Press and Lior Wolf. "Using the Output Embedding to Improve Language Models." In: *Proceedings of the 15th Conference of the European Chapter of the Association for Computational Linguistics, EACL 2017, Valencia, Spain, April 3-7, 2017, Volume 2: Short Papers*. Ed. by Mirella Lapata, Phil Blunsom, and Alexander Koller. Association for Computational Linguistics, 2017, pp. 157–163. URL: https://arxiv.org/pdf/1608.05859.

[26] Longlong Jing and Yingli Tian. "Self-Supervised Visual Feature Learning With Deep Neural Networks: A Survey." In: *IEEE Trans. Pattern Anal. Mach. Intell.* 43.11 (2021), pp. 4037–4058. URL: https://arxiv.org/pdf/1902.06162.

[27] Jingzhao Zhang et al. "Why are Adaptive Methods Good for Attention Models?" In: *Advances in Neural Information Processing Systems*. Ed. by H. Larochelle et al. Vol. 33. Curran Associates, Inc., 2020, pp. 15383–15393. URL: https://proceedings.neurips.cc/paper_files/paper/2020/file/b05b57f6add810d3b7490866d74c0053-Paper.pdf.

[28] Kaiming He et al. "Deep Residual Learning for Image Recognition." In: *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*. June 2016. URL: https://arxiv.org/pdf/1512.03385.

[29] Olga Kovaleva et al. "Revealing the Dark Secrets of BERT." In: *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing, EMNLP-IJCNLP 2019, Hong Kong, China, November 3-7, 2019*. Ed. by Kentaro Inui et al. Association for Computational Linguistics, 2019, pp. 4364–4373. URL: https://arxiv.org/pdf/1908.08593.

[30] Elena Voita et al. "Analyzing Multi-Head Self-Attention: Specialized Heads Do the Heavy Lifting, the Rest Can Be Pruned." In: *Proceedings of the 57th Conference of the Association for Computational Linguistics, ACL 2019, Florence, Italy, July 28- August 2, 2019, Volume 1: Long Papers*. Ed. by Anna Korhonen, David R. Traum, and Lluis Marquez. Association for Computational Linguistics, 2019, pp. 5797–5808. URL: https://arxiv.org/pdf/1905.09418.

[31] Adam Paszke et al. "PyTorch: An Imperative Style, High-Performance Deep Learning Library." In: *Advances in Neural Information Processing Systems 32: Annual Conference on Neural Information Processing Systems 2019, NeurIPS 2019, December 8-14, 2019, Vancouver, BC, Canada*. Ed. by Hanna M. Wallach et al. 2019, pp. 8024–8035. URL: https://arxiv.org/pdf/1912.01703.

Preprints

[32] OpenAI. *GPT-4 Technical Report*. 2024. URL: https://arxiv.org/pdf/2303.08774.

[33] Zhen Yang et al. *GPT Can Solve Mathematical Problems Without a Calculator*. 2023. URL: https://arxiv.org/abs/2309.03241.

[34] Tiedong Liu and Bryan Kian Hsiang Low. *Goat: Fine-tuned LLaMA Outperforms GPT-4 on Arithmetic Tasks*. 2023. URL: https://arxiv.org/abs/2305.14201.

[35] Yuntian Deng, Yejin Choi, and Stuart Shieber. *From Explicit CoT to Implicit CoT: Learning to Internalize CoT Step by Step*. 2024. URL: https://arxiv.org/abs/2405.14838.

[36] Ruoqi Shen et al. *Positional Description Matters for Transformers Arithmetic*. 2023. URL: https://arxiv.org/abs/2311.14737.

[37] Samy Jelassi et al. *Length Generalization in Arithmetic Transformers*. 2023. URL: https://arxiv.org/pdf/2306.15400.

[38] Alec Radford et al. *Improving language understanding by generative pre-training*. 2018. URL: https://cdn.openai.com/research-covers/language-unsupervised/language_understanding_paper.pdf.

[39] Alec Radford et al. *Language Models are Unsupervised Multitask Learners*. 2018. URL: https://cdn.openai.com/better-language-models/language_models_are_unsupervised_multitask_learners.pdf.

[40] Randall Balestriero et al. *A Cookbook of Self-Supervised Learning*. 2023. URL: https://arxiv.org/abs/2304.12210.

[41] Dan Hendrycks and Kevin Gimpel. *Gaussian Error Linear Units (GELUs)*. 2023. URL: https://arxiv.org/abs/1606.08415.

Other Links

[42] OpenAI. *Introducing ChatGPT*. Nov. 2022. URL: https://openai.com/blog/chatgpt/ (visited on Dec. 19, 2023).

[43] OpenGenus. *GPT-2 vs GPT-3 vs GPT-3.5 vs GPT-4: A Comprehensive Comparison of OpenAI LLMs*. 2023. URL: https://iq.opengenus.org/gpt2-vs-gpt3-vs-gpt35-vs-gpt4/ (visited on Dec. 21, 2023).

[44] Ethan Mollick. *ChatGPT Is a Tipping Point for AI*. Harvard Business Review. Dec. 14, 2022. URL: https://hbr.org/2022/12/chatgpt-is-a-tipping-point-for-ai (visited on Oct. 30, 2025).

[45] Jay Alammar. *The Illustrated Transformer*. URL: http://jalammar.github.io/illustrated-transformer/ (visited on June 6, 2023).

[46] Hugging Face. *Summary of the tokenizers*. 2024. URL: https://huggingface.co/docs/transformers/tokenizer_summary (visited on Jan. 4, 2024).

[47] Hugging Face. *OpenAI GPT2 - Hugging Face Transformers Documentation*. 2024. URL: https://huggingface.co/docs/transformers/model_doc/gpt2 (visited on Jan. 4, 2024).

[48] Jay Alammar. *The Illustrated GPT2*. URL: http://jalammar.github.io/illustrated-gpt2/ (visited on June 6, 2023).

[49] Lei Mao. *Autoregressive Model and Autoregressive Decoding for Sequence to Sequence Tasks*. 2021. URL: https://leimao.github.io/blog/Autoregressive-Model-Autoregressive-Decoding/ (visited on Jan. 4, 2024).

[50] OpenAI. *Better language models and their implications*. 2019. URL: https://openai.com/research/better-language-models (visited on Jan. 8, 2024).

[51] Hugging Face. *Pretrained models*. URL: https://huggingface.co/transformers/v2.2.0/pretrained_models.html (visited on Jan. 11, 2024).

[52] Andrej Karpathy. *nanoGPT*.
`GitHub repository`. 2023. URL: https://github.com/karpathy/nanoGPT (visited on Oct. 19, 2025).

[53] Hugging Face (GitHub). *Transformers*.
`Commit ID: 7631db0fdcfbd95b1f21d8034a0b8df73b9380ff`. 2018. URL: https://github.com/huggingface/transformers.